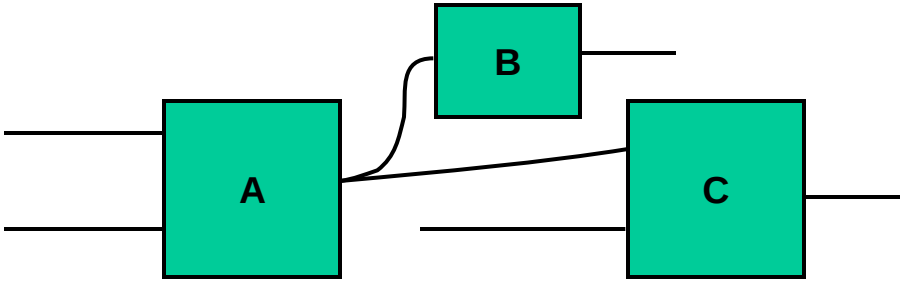
CAD for VLSI

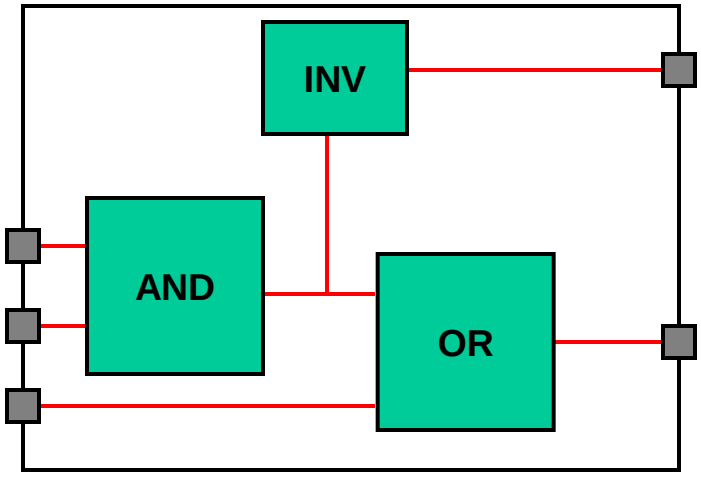
Routing 1

Introduction to Routing

Routing in Design Flow



Netlist



Routing

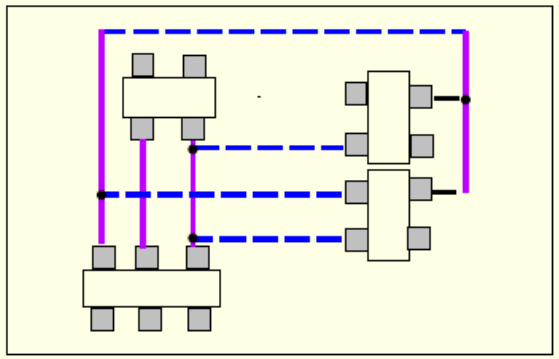
Floorplan/Placement

The Routing Problem

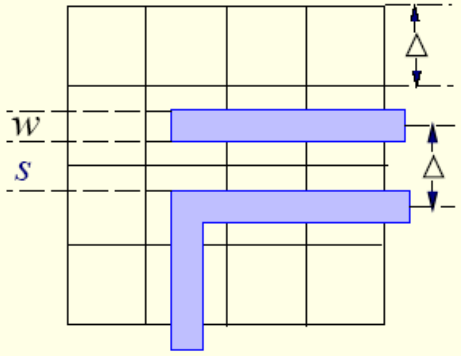
- Apply it after floorplanning/placement
- Input:
  - Netlist
  - Timing budget for critical nets (typically)
  - Locations of blocks and pins
- Output:
  - Geometric layouts of all nets
- Objective:
  - Minimize the total wire length, the number of vias, or just completing all connections without increasing the chip area.
  - Each net meets its timing budget

The Routing Constraints

- Examples:
  - Placement constraint
  - Number of routing layers
  - Delay constraint
  - Meet all geometrical constraints (design rules)
  - Physical/Electrical/Manufacturing constraints:
    - Crosstalk
    - Process variations, yield, or lithography issues?



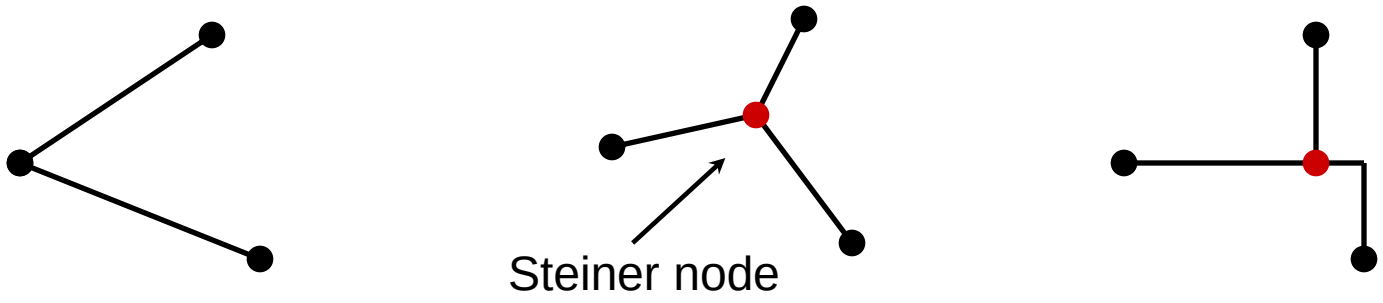
Two−layer routing



Geometrical constraint

Steiner Tree

- For a multi-terminal net, we can construct a spanning tree to connect all the terminals together
  - The wire length will be large
- Better use Steiner Tree:
  - A tree connecting all terminals and some additional nodes (Steiner nodes)
- Rectilinear Steiner Tree:
  - Steiner tree in which all the edges run horizontally and vertically



Routing Problem is Very Hard

- Minimum Spanning Tree Problem:
  - Use Prim's or Kruskal Algorithm
  - This is a P problem. Easy!

- Minimum Steiner Tree Problem:
  - Given a net, find the Steiner tree with the minimum length
  - This problem is NP-Complete!

- May need to route tens of thousands of nets simultaneously without overlapping
- Obstacles may exist in the routing region

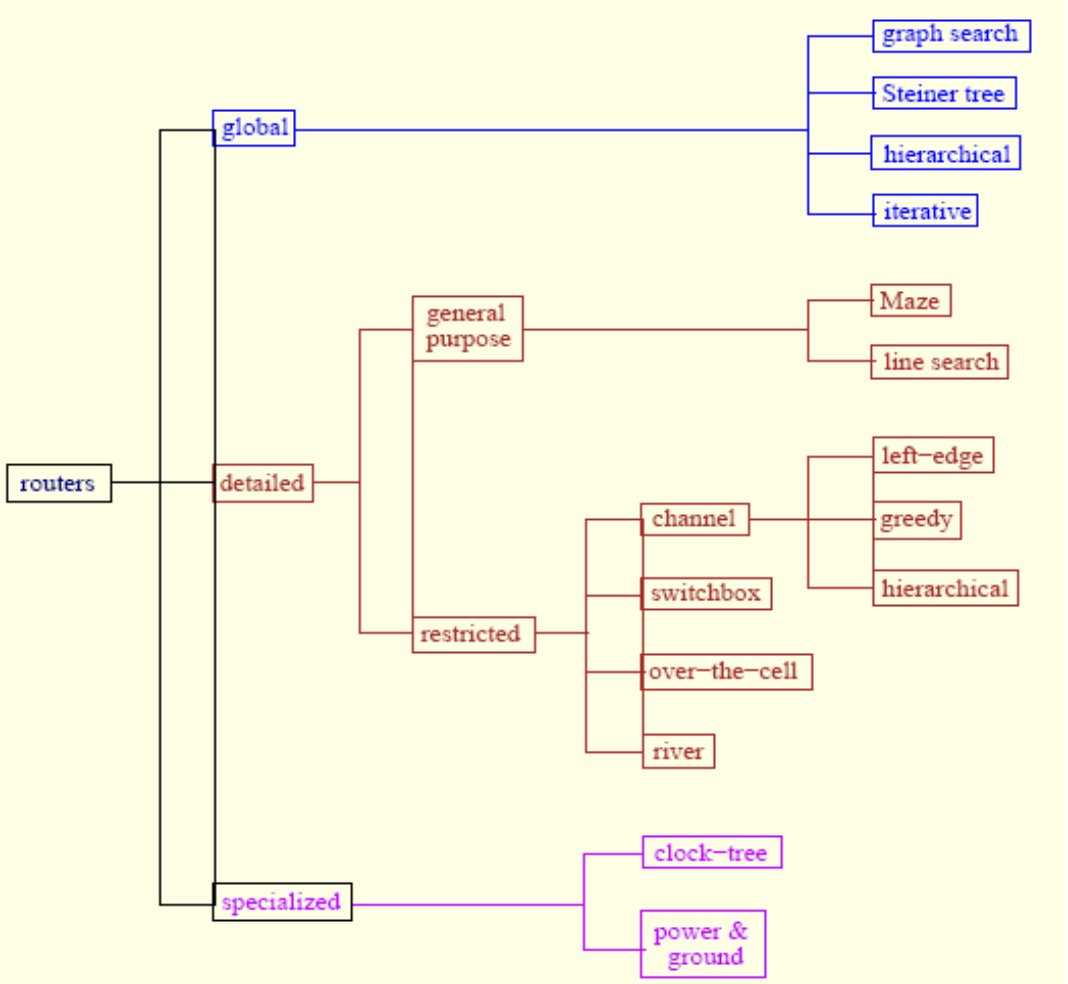
Routing Category

- Global Routing
- Detailed Routing
  - Channel
  - Switchbox
- Others:
  - Maze routing
  - Over the cell routing
  - Clock routing
  - Re-distribution layer (RDL) routing
  - Package routing

Approaches for Routing

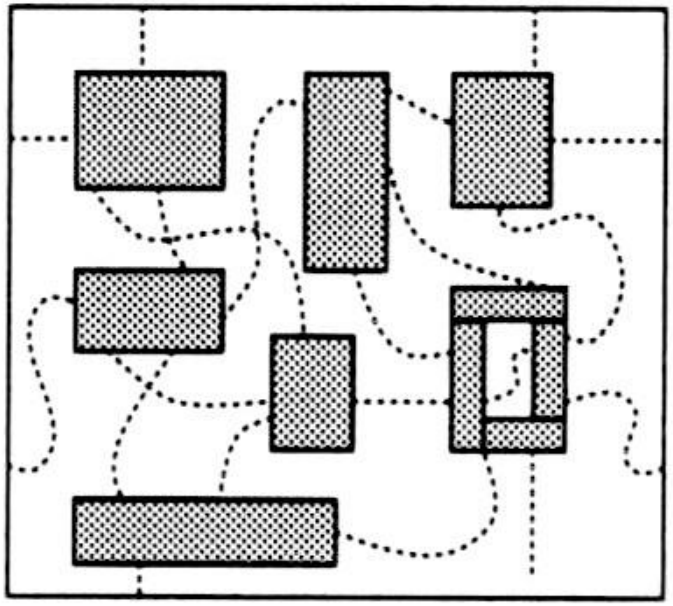
- Sequential Approach:
  - Route the nets one at a time
  - Order depends on factors like criticality, estimated wire length, and number of terminals
  - When further routing of nets is not possible because some nets are blocked by nets routed earlier, apply 'Rip-up and Reroute' technique (or 'Shove-aside' technique)
- Concurrent Approach:
  - Consider all nets simultaneously, i.e., no ordering
  - Can be formulated as integer programming

Classification of Routing

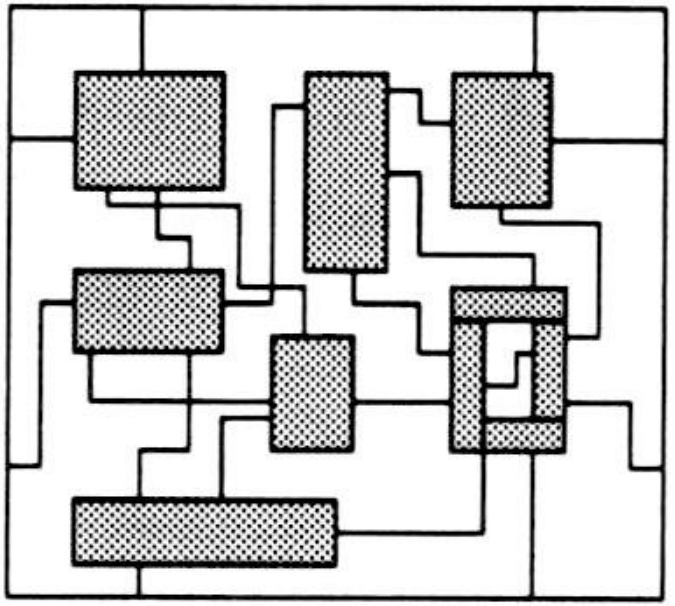


General Routing Paradigm

Two phases:



Global Routing



Detailed Routing

Extraction and Timing Analysis

- After global routing and detailed routing, information of the nets can be extracted and delays can be analyzed
- If some nets fail to meet their timing budget, detailed routing and/or global routing needs to be repeated

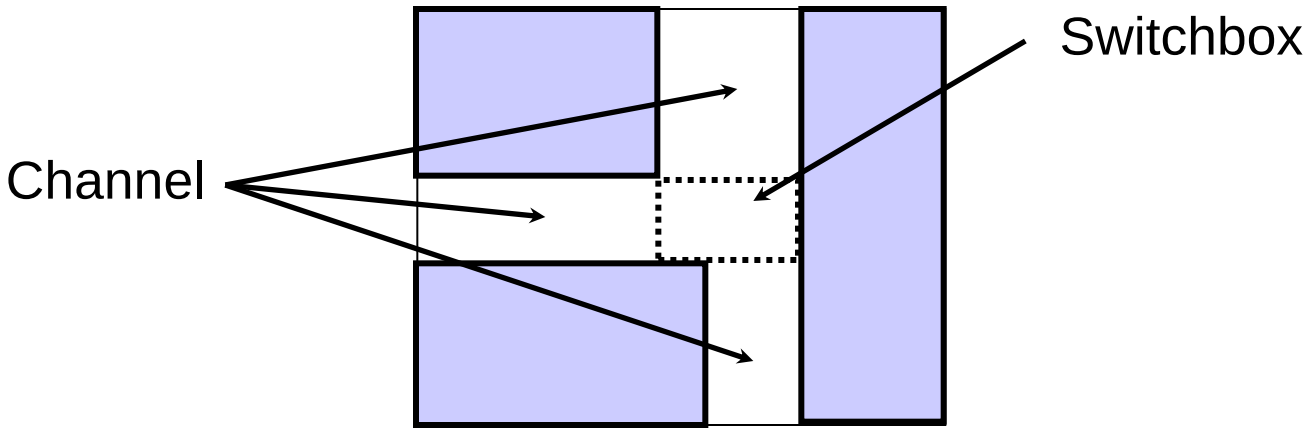
Global Routing

Global routing is divided into 3 phases:

1. Region definition
2. Region assignment
3. Pin assignment to routing regions

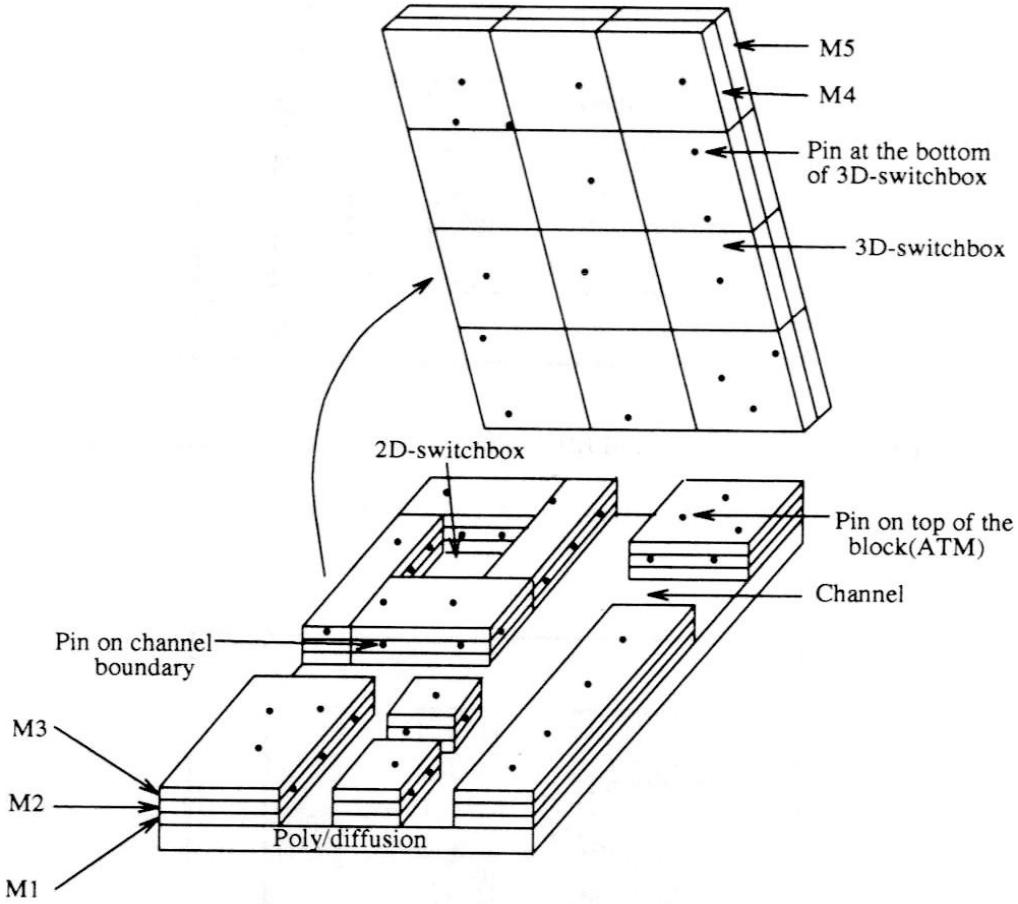
Region Definition

- Divide the routing area into routing regions of simple shape (rectangular):



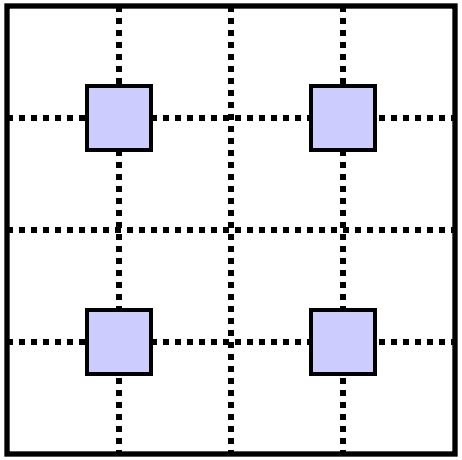
  - Channel: Pins on 2 opposite sides
  - 2-D Switchbox: Pins on 4 sides
  - 3-D Switchbox: Pins on all 6 sides

Routing Regions

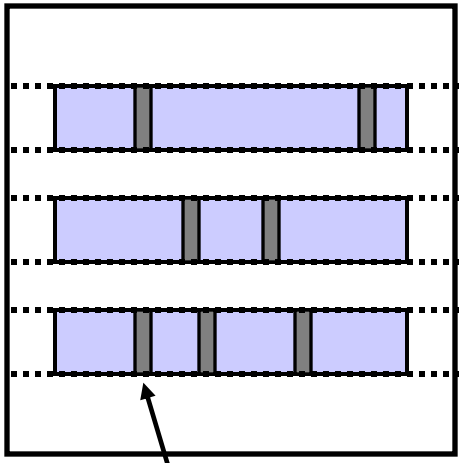


Routing Regions in Different Design Styles

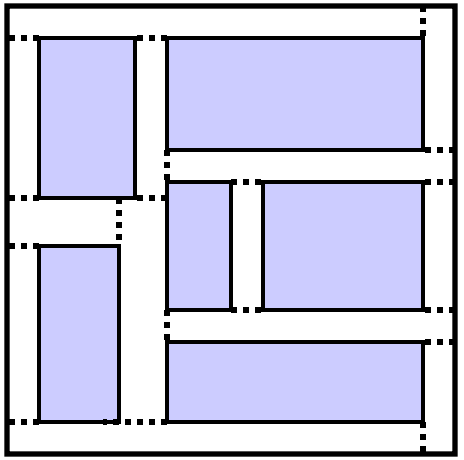
Gate-Array



Standard-Cell

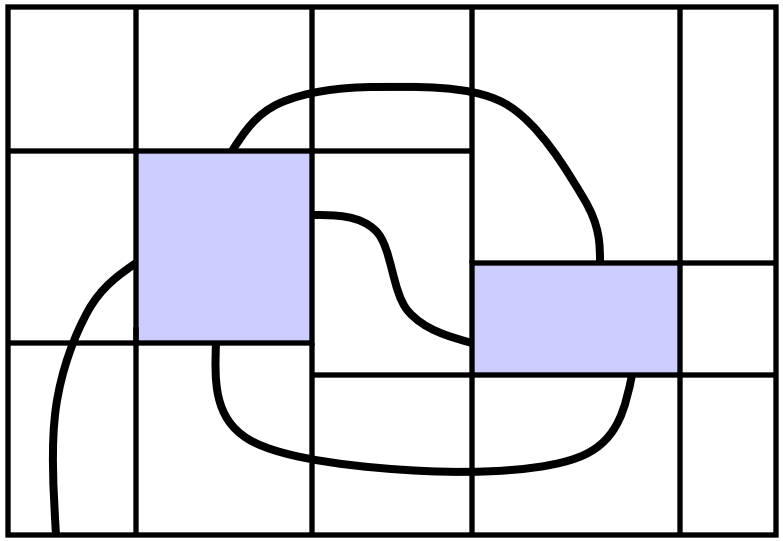


Full-Custom



Region Assignment

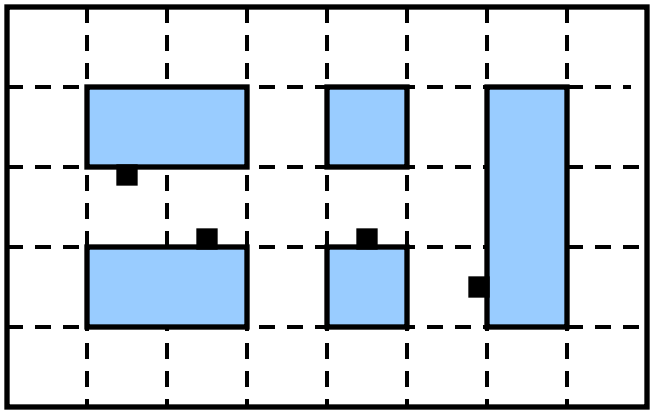
- Assign routing regions to each net.
- Need to consider timing budget of nets and routing congestion of the regions



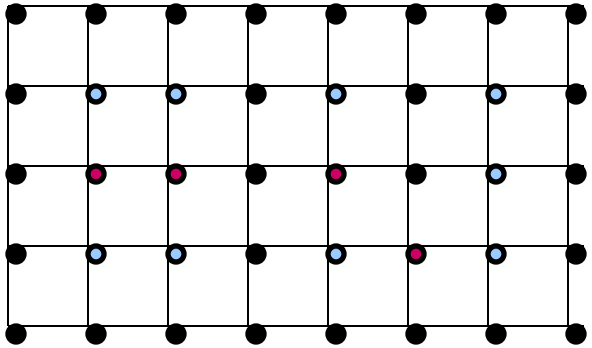
Graph Modeling of Routing Regions

- Grid Graph Modeling
- Checker Board Graph Modeling
- Channel Intersection Graph Modeling

Grid Graph

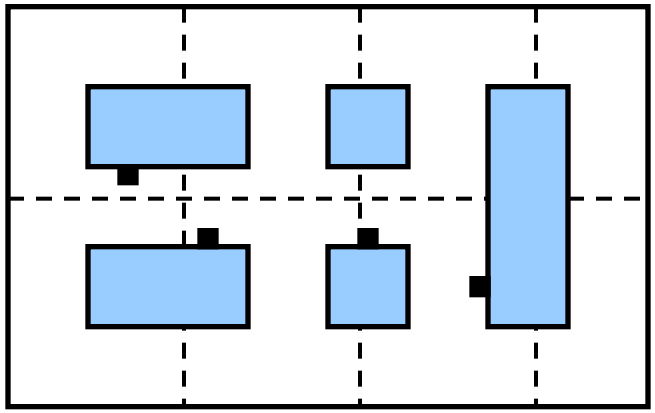


■ A terminal

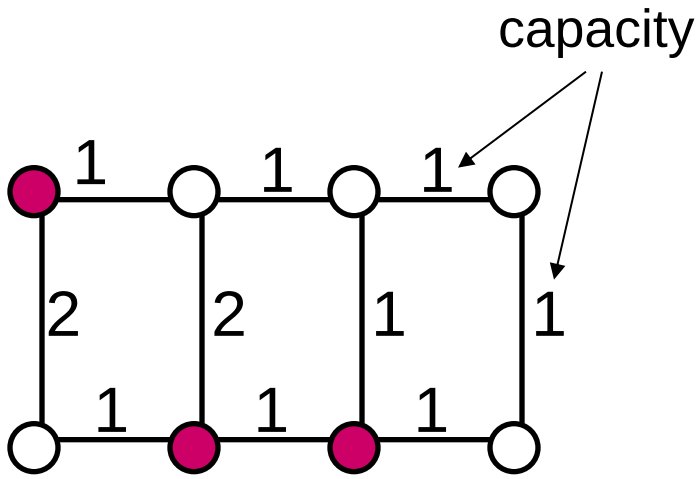


• A node with terminals

Checker Board Graph

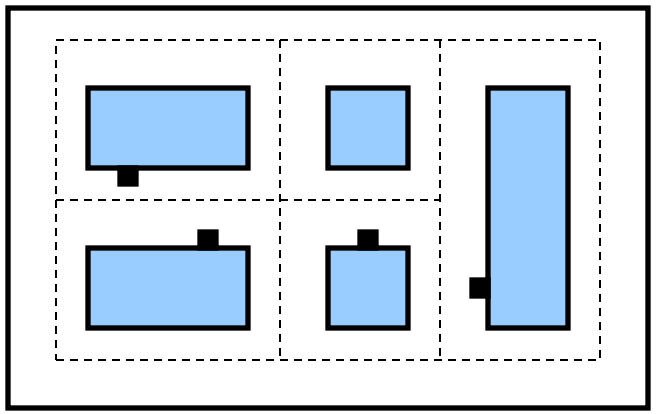


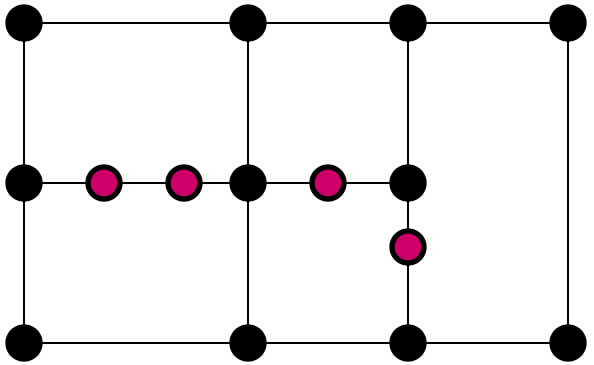
■ A terminal



A node with terminals

Channel Intersection Graph





■ A terminal

● A node with terminals

Routings along the channels

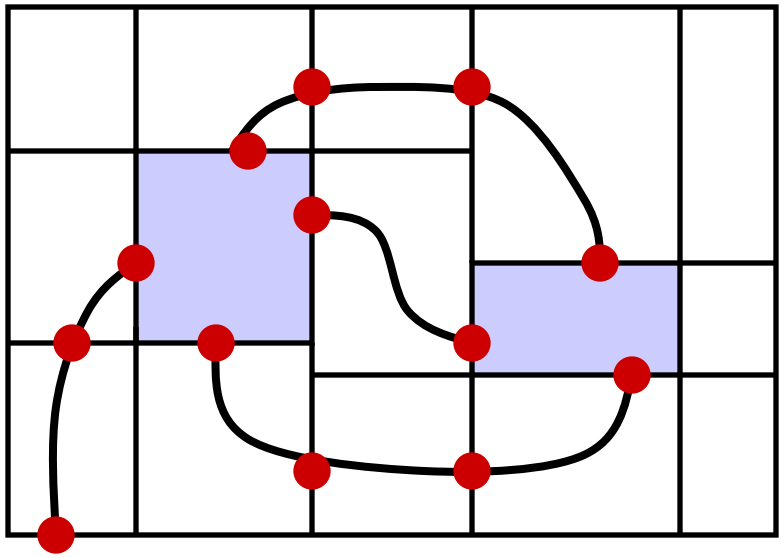
Approaches for Global Routing

- Sequential Approach:
  - Route the nets one at a time
  - Order dependent on factors like criticality, estimated wire length, etc.
  - If further routing is impossible because some nets are blocked by nets routed earlier, apply Rip-up and Reroute technique
  - This approach is much more popular

- Concurrent Approach:
  - Consider all nets simultaneously
  - Can be formulated as an integer program

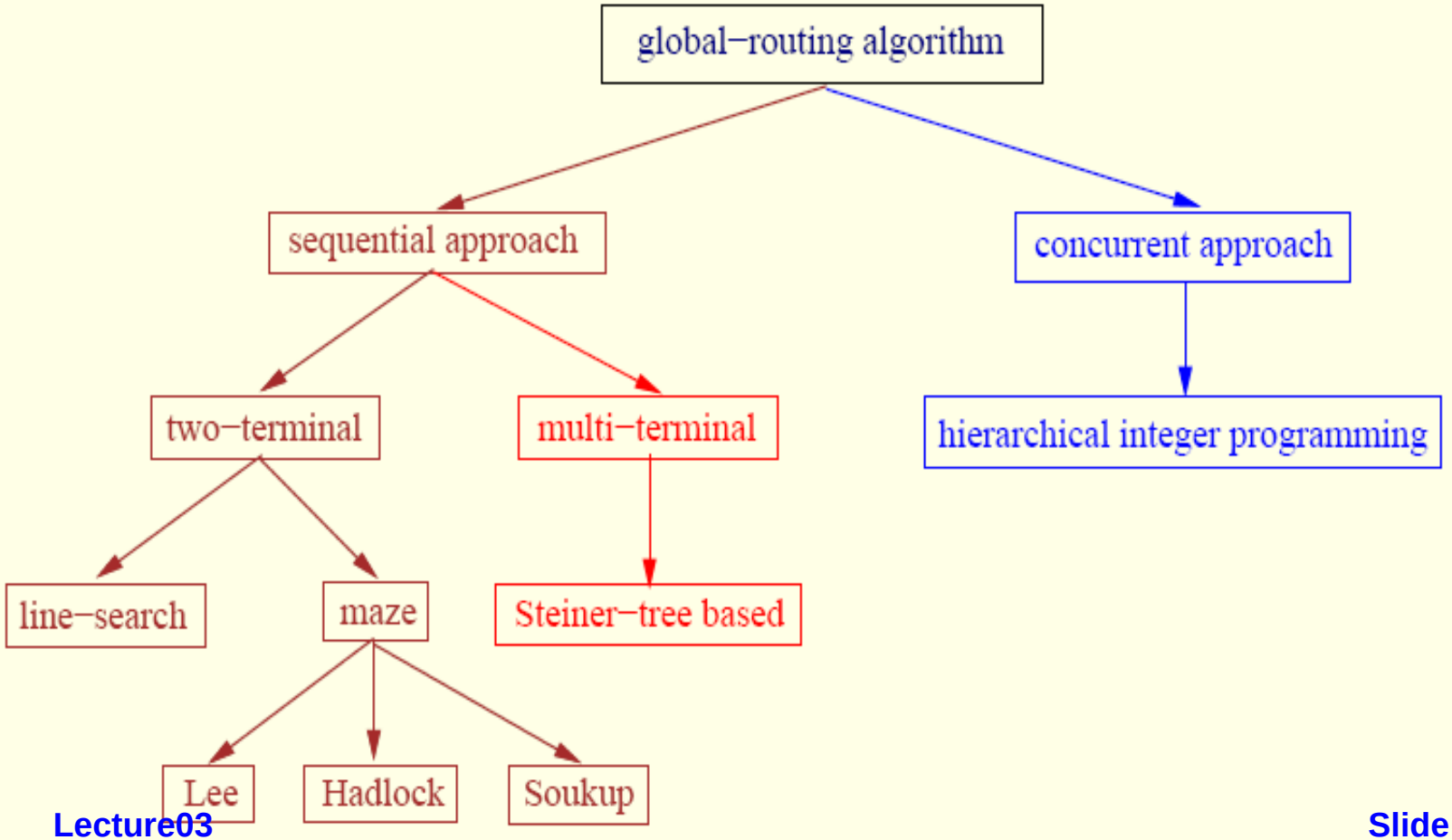
Pin Assignment

- Assign pins on routing region boundaries for each net (Prepare for the detailed routing stage for each region)



Classification of Global-Routing Algorithm

- **Sequential approach:** Assigns priority to nets; routes one net at a time based on its priority (net ordering?).
- **Concurrent approach:** All nets are considered at the same time (complexity?)

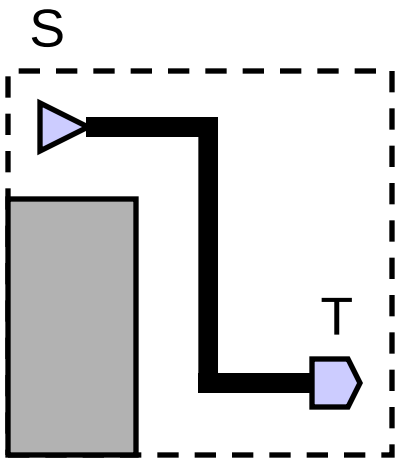


Maze Routing

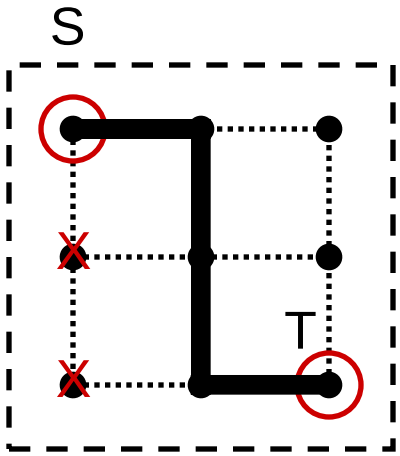
Maze Routing Problem

- Given:
  - A planar rectangular grid graph
  - Two points S and T on the graph
  - Obstacles modeled as blocked vertices
- Objective:
  - Find the shortest path connecting S and T
- This technique can be used in global or detailed routing (switchbox) problems

Grid Graph



Area Routing

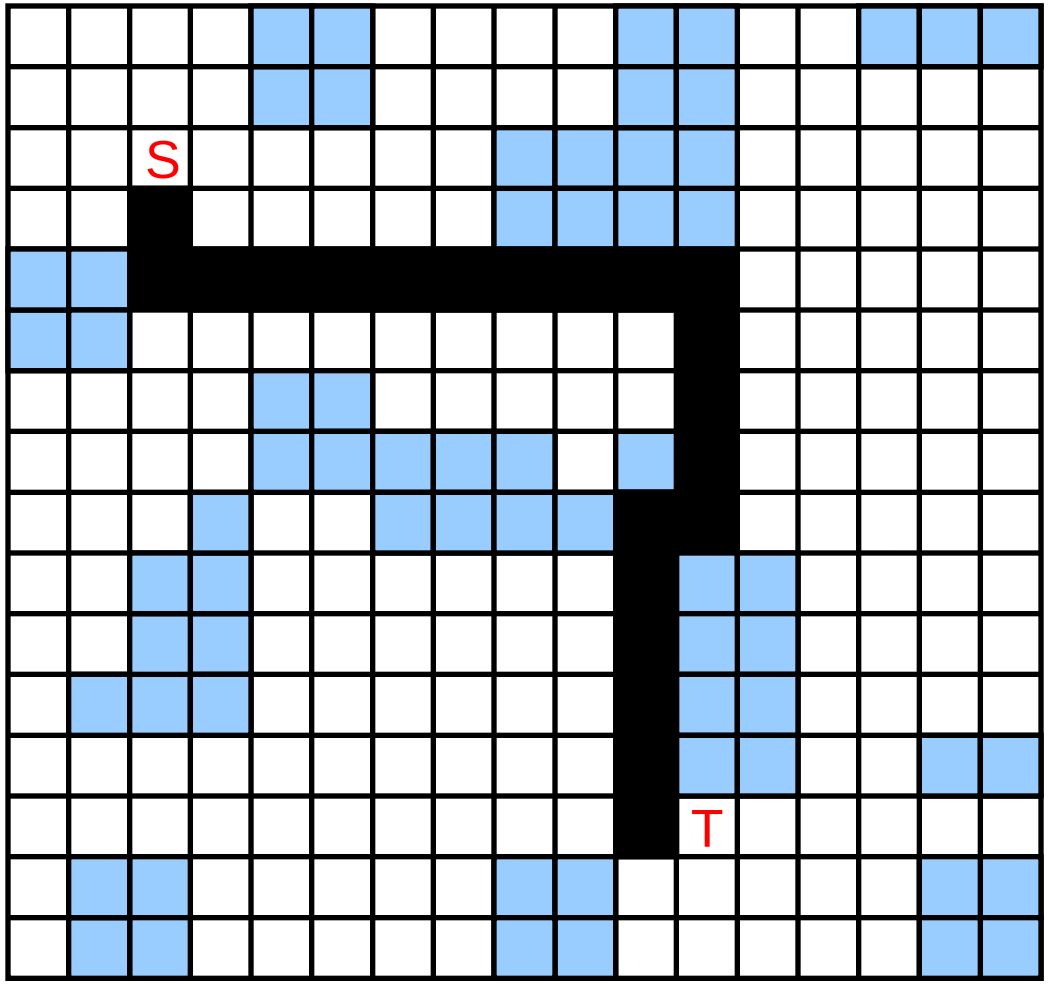


Grid Graph
(Maze)

S	✓	
X	✓	
X	✓	T

Simplified
Representation

Maze Routing



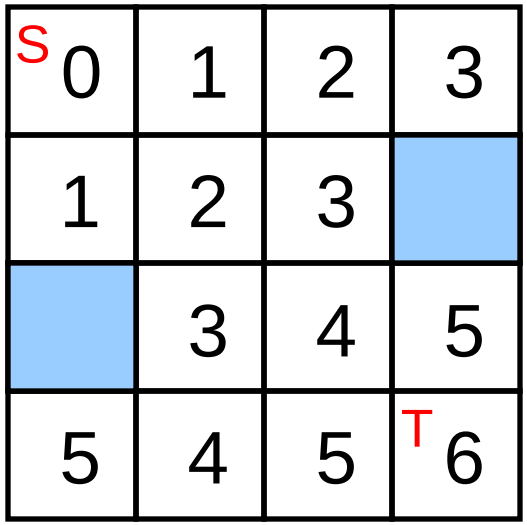
Lee's Algorithm

"An Algorithm for Path Connection and its Application" , C.Y. Lee, IRE Transactions on Electronic Computers, 1961.

Basic Idea

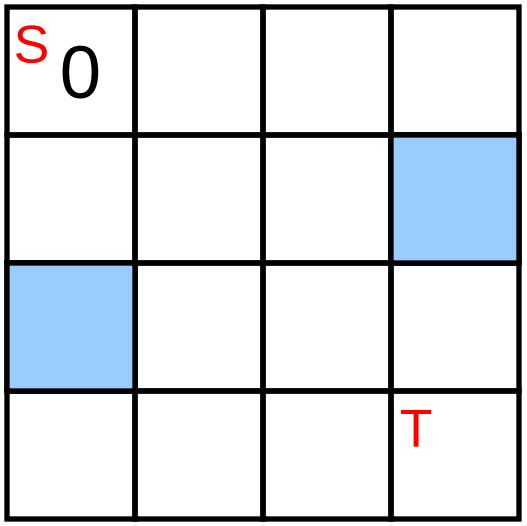
- A Breadth-First Search (BFS) of the grid graph
- Always find the shortest path
- Consists of two phases:
  - Wave Propagation
  - Retrace

An Illustration

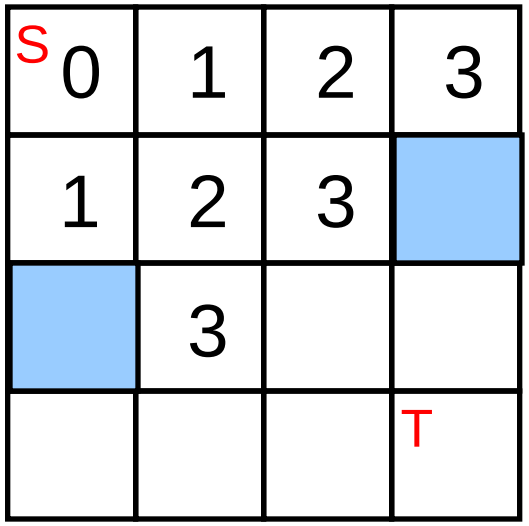


Wave Propagation

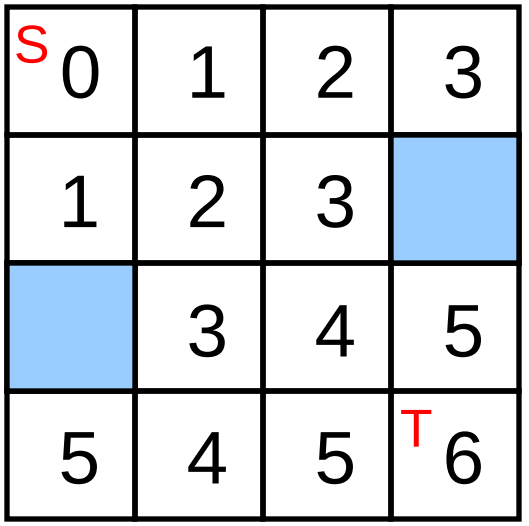
- At step k, all vertices at Manhattan-distance k from S are labeled with k
- A Propagation List (FIFO) is used to keep track of the vertices to be considered next



After Step 0



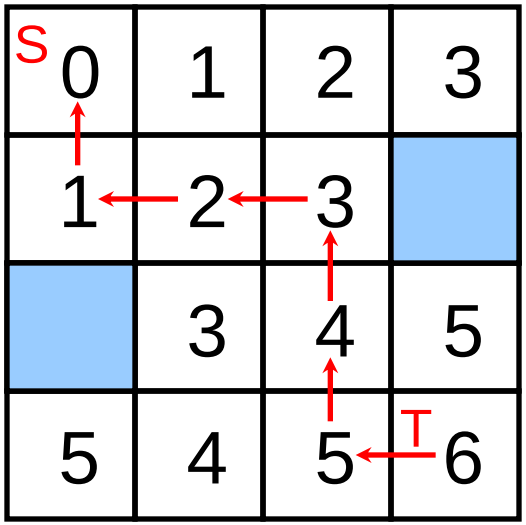
After Step 3



After Step 6

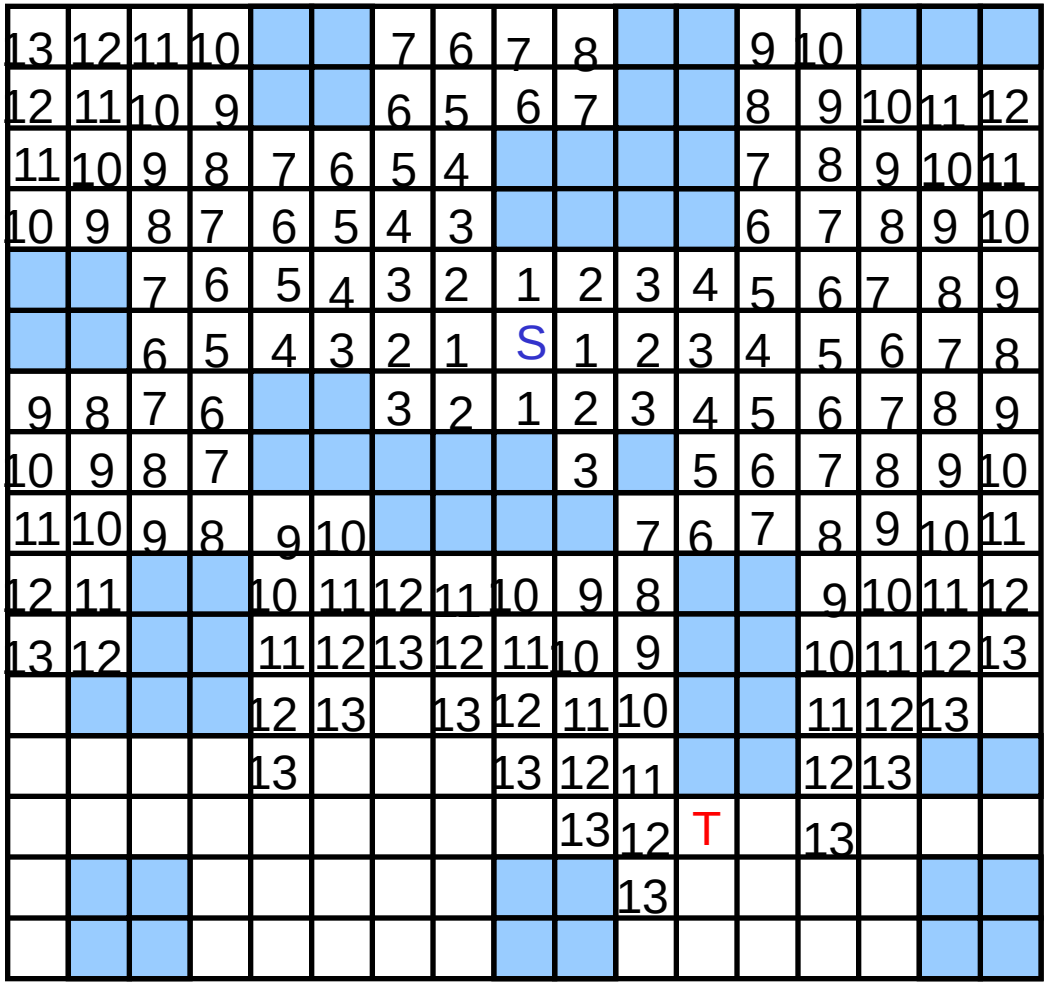
Retrace

- Trace back the actual route
- Starting from *T*
- At *vertex with k, go to any vertex with label k*-1



Final labeling

How many grids visited using Lee's algorithm?



Time and Space Complexity

- For a grid structure of size $w \times h$:
  - Time per net = O(wh)
  - Space = O(wh log wh)
    - O(log wh) bits are needed to store each label.
- For a 4000 $\times$ 4000 grid structure:
  - 24 bits per label
  - Total 48 Mbytes of memory!

Improvement to Lee's Algorithm

- Improvement on memory:
  - Aker's Coding Scheme
- Improvement on run time:
  - Starting point selection
  - Double fan-out
  - Framing
  - Hadlock's Algorithm
  - Soukup's Algorithm

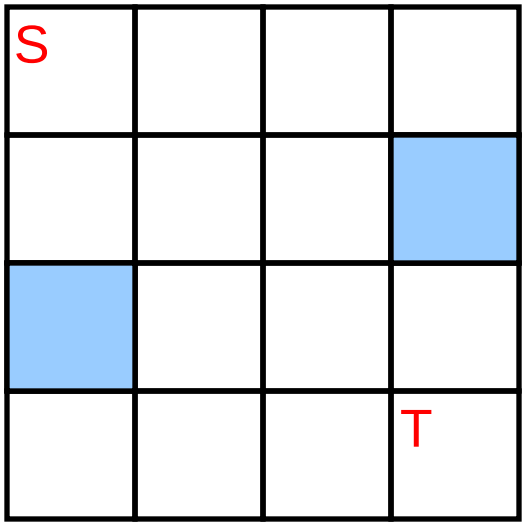
Aker's Coding Scheme to Reduce Memory Usage

Aker's Coding Scheme

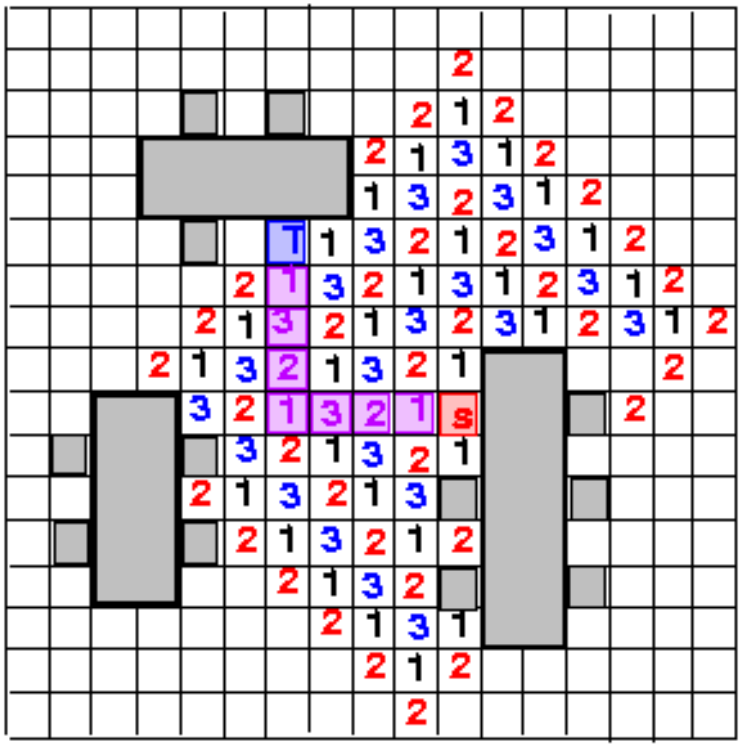
- For the Lee's algorithm, labels are needed during the retrace phase
- But there are only two possible labels for neighbors of each vertex labeled *i*, which are, *i*-1 and *i*+1
- So, is there any method to reduce the memory usage?

Aker's Coding Scheme

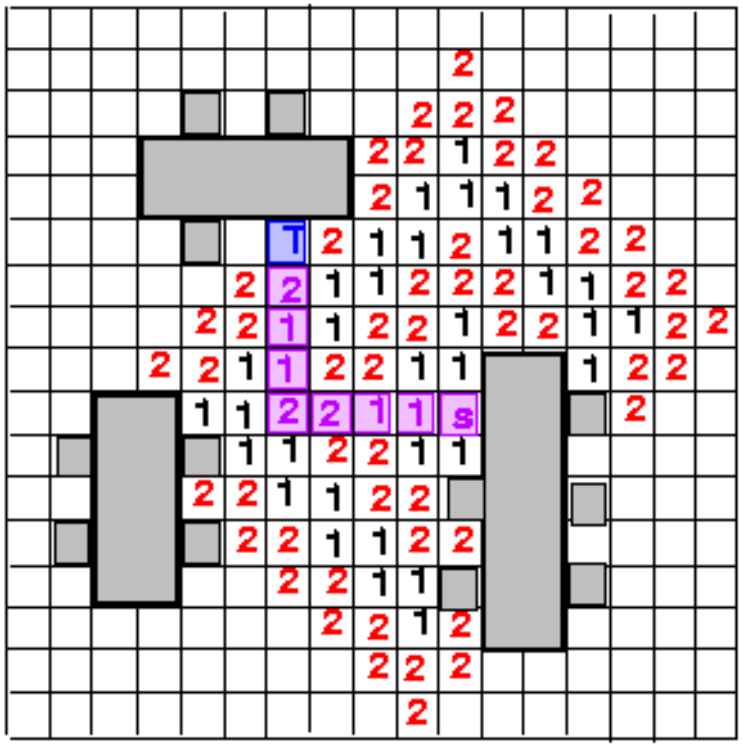
- One bit (independent of grid size) is enough to distinguish between the two labels
- Sequence: … (what sequence can be applied?)
  - The labels before and after each label must be different in order to tell the forward or the backward directions
- How to represent empty / blocked ?



Aker's Coding Example



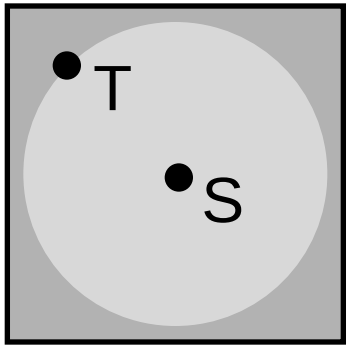
Sequence: 1, 2, 3, 1, 2, 3, ...

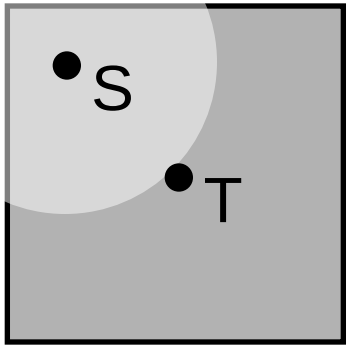


Sequence: 1, 1, 2, 2, 1, 1, 2, 2, ...

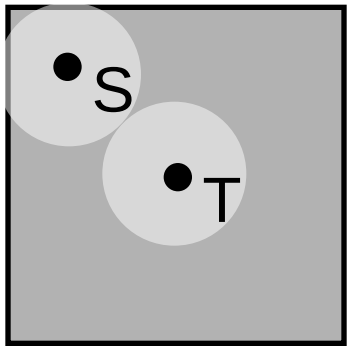
Schemes to Reduce Run Time

Starting Point Selection

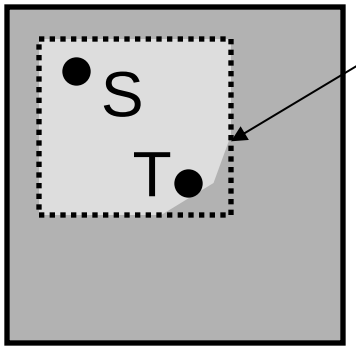




Double Fan-Out



Framing



The frame can be 10-20% larger than the bounding box containing *S* and *T*. Redo if the search fails.

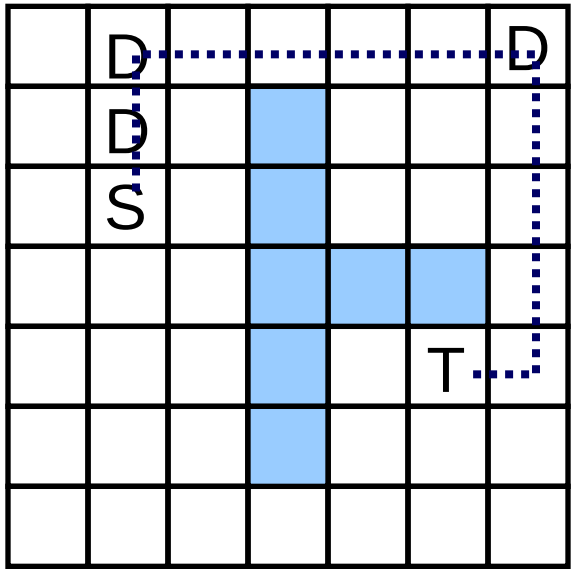
Hadlock's Algorithm to Reduce Run Time

Detour Number

- For a path P from S to T, let detour number d(P) = # of grids directed away from T, then
  - L(P) = MD(S,T) + 2d(P)

length

shortest Manhattan distance



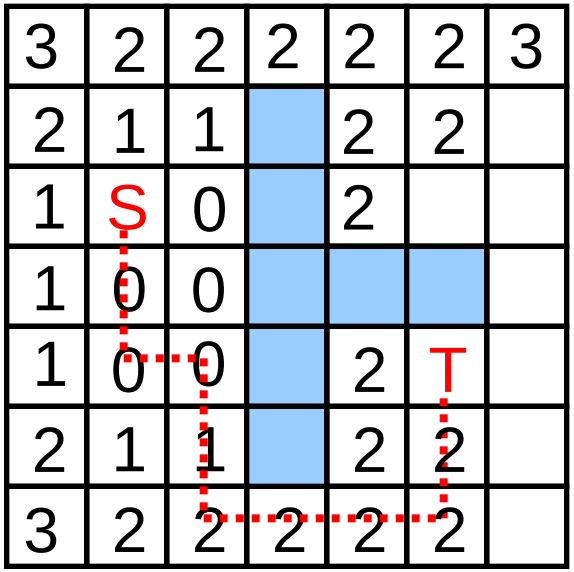
D: Detour

d(P) = 3
MD(S,T) = 6
L(P) = 6+2x3 = 12

- So minimizing L(P) and d(P) are the same

Hadlock's Algorithm

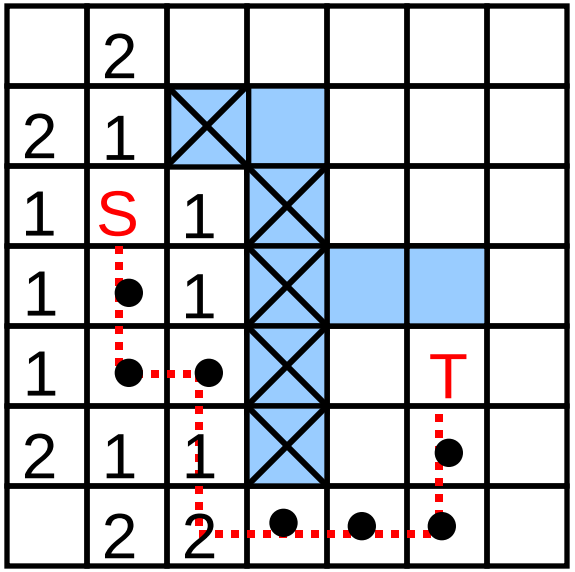
- Label vertices with detour numbers
- Vertices with smaller detour number are expanded first
- Therefore, favor paths without detour



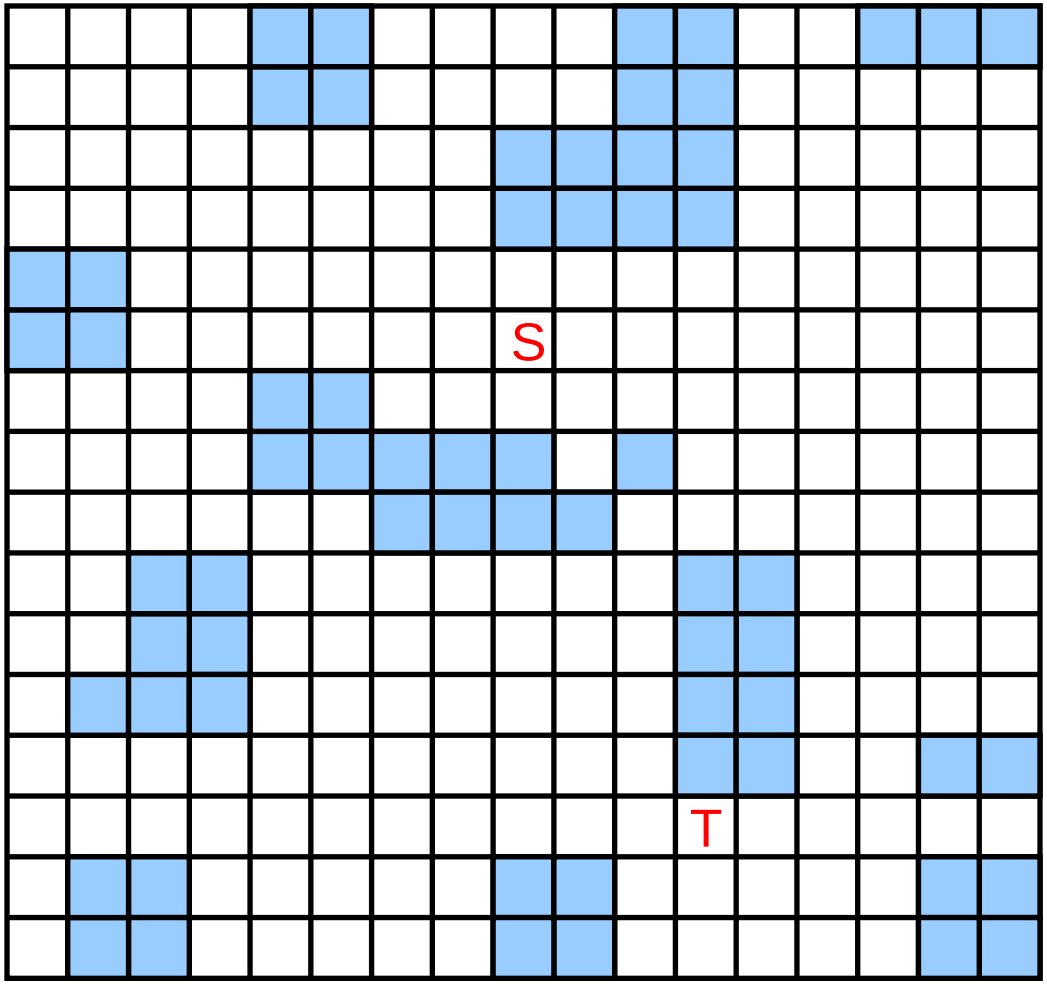
Soukup's Algorithm to Reduce Run Time

Basic Idea

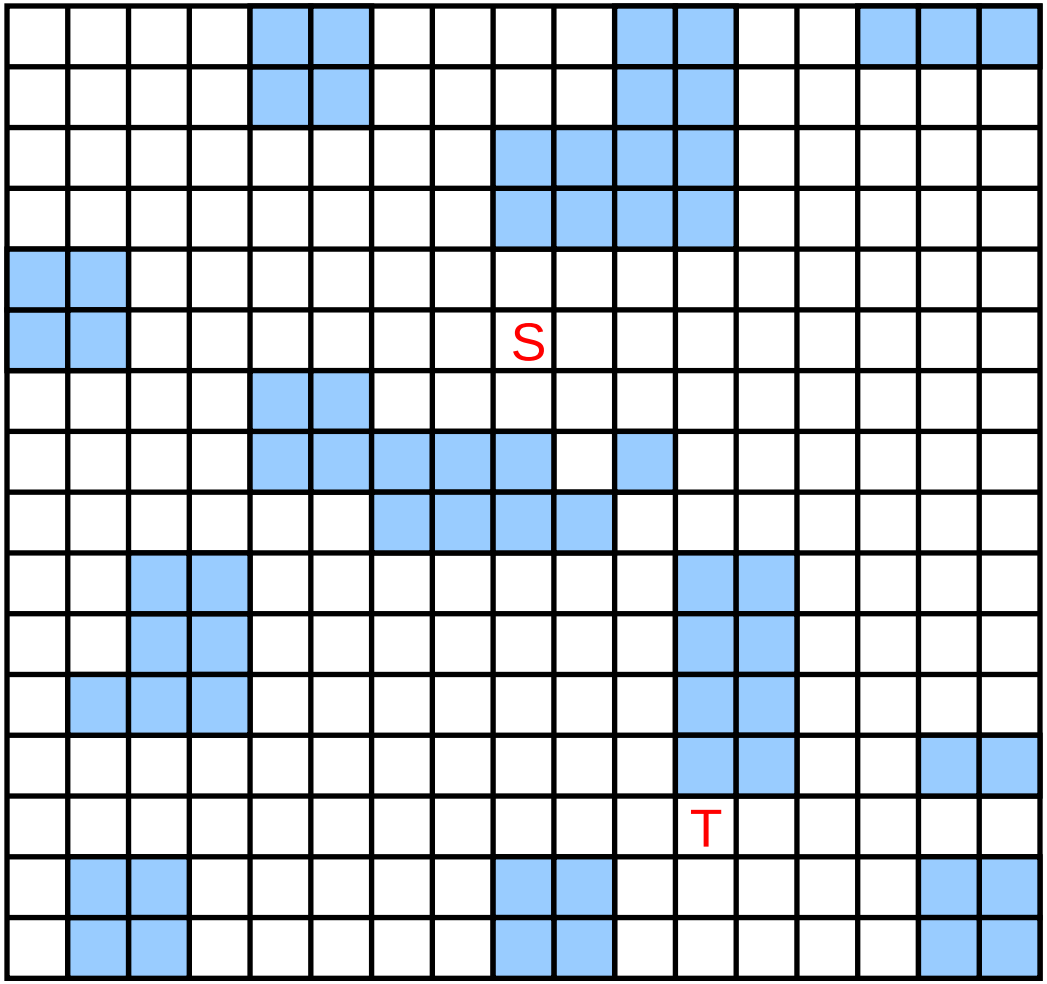
- Soukup's Algorithm: DFS+BFS
  - Explore in the direction towards the target without changing direction. (DFS)
  - If obstacle is hit, search around the obstacle. (BFS)
- May get Sub-Optimal solution



How many grids visited using Hadlock's?



How many grids visited using Soukup's?



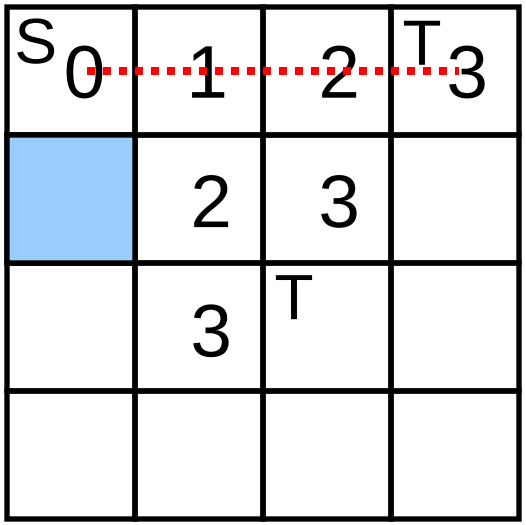
Multi-Terminal Nets

- For a *k*-terminal net, connect the *k* terminals using a rectilinear Steiner tree with the shortest wire length on the maze
- This problem is NP-Complete
- Just want to find some good heuristics

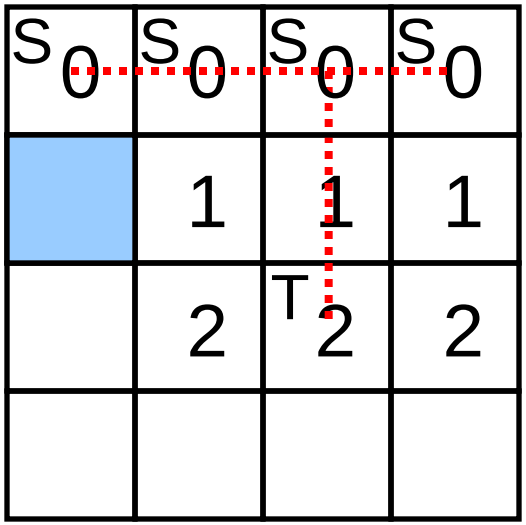
Multi-Terminal Nets

- This problem can be solved by extending the Lee's algorithm:
  - Connect one terminal at a time, or
  - Search for several targets simultaneously, or
  - Propagate wave fronts from several different sources simultaneously

Extension to Multi-Terminal Nets



1st Iteration



2nd Iteration

Global Routing

Global Routing Approaches

- Sequential Approach (Rip-up and Re-route)
  - Maze Routing
  - Line probing
  - Shortest Path Based Algorithms
  - Steiner Tree Based Algorithms
- Concurrent Approach
  - Integer Programming

Sequential Approach

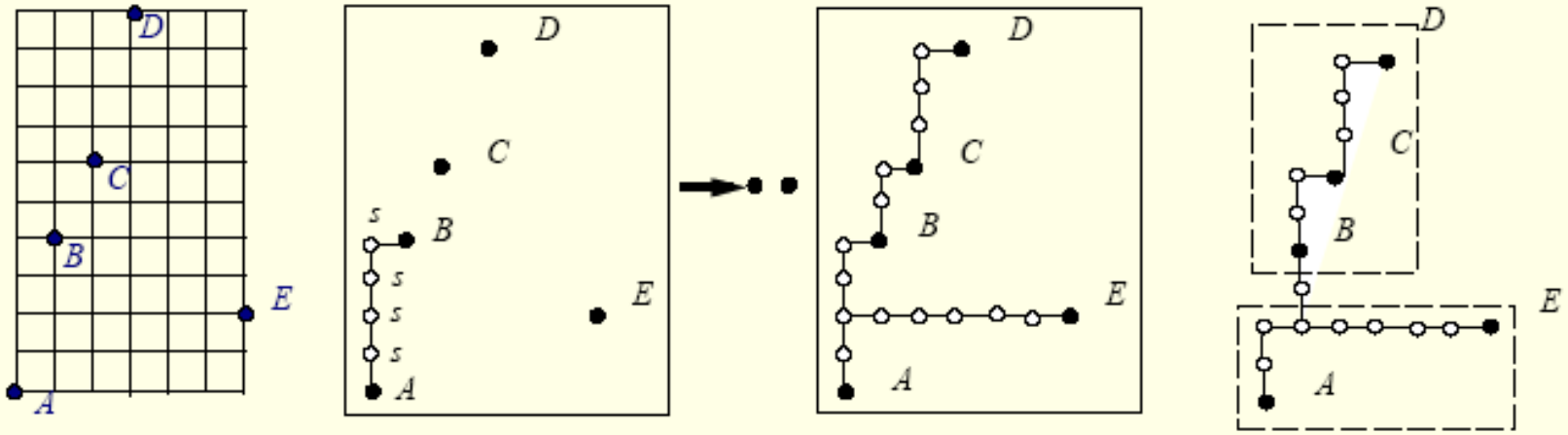
- Algorithm:

1. Graph modeling of the routing regions
2. For each net k:
   - 2.1 Find a route r for net k on the graph.
   - 2.2 For each edge e in r:
     - 2.2.1 capacity(e) = capacity(e) - 1
     - 2.2.2 if capacity(e) < 0 then cost(e) = $\alpha \times$ cost(e)

We can use different methods to do this

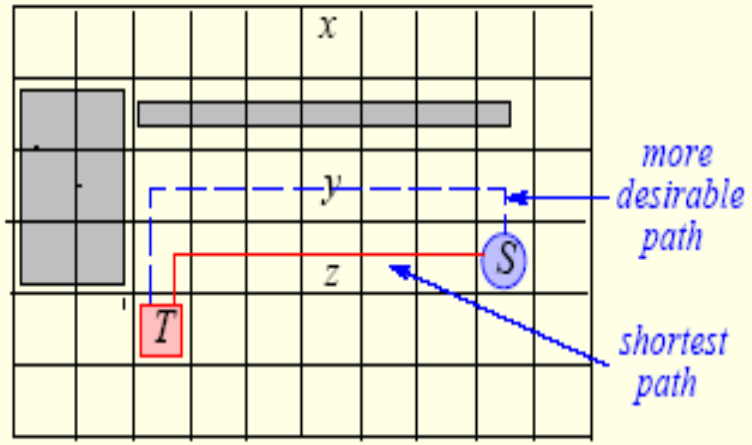
Maze Routing for Multi-Terminal Nets

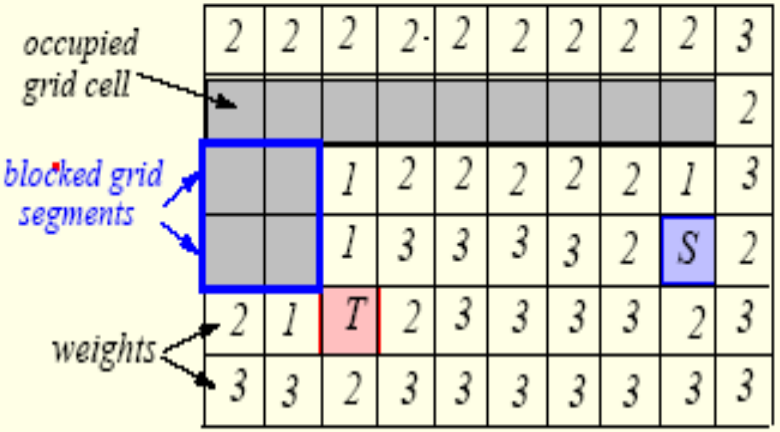
- Step 1: Propagate wave from the source s to the closet target.
- Step 2: Mark ALL cells on the path as s.
- Step 3: Propagate wave from ALL s cells to the other cells.
- Step 4: Continue until all cells are reached.
- Step 5: Apply heuristics to further reduce the tree cost.



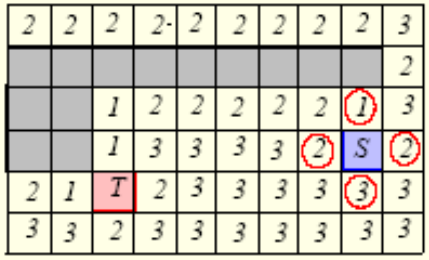
Maze Routing on Weighted Graph

- Motivation: finding more desirable paths
- $weight(grid\ cell) = \#\ of\ unblocked\ grid\ cell\ segments - 1$

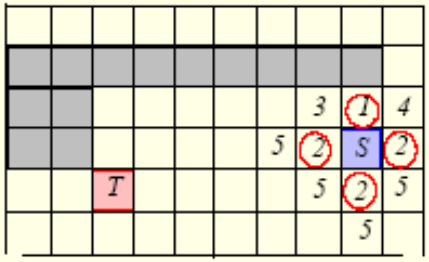




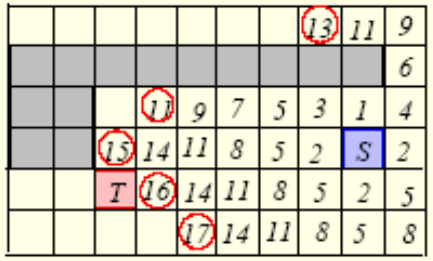
Example of Weighted Graph

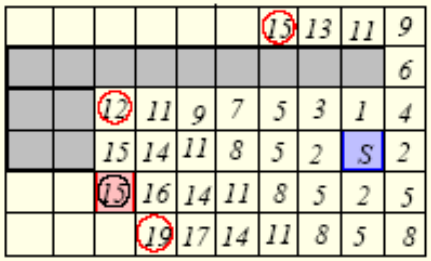


initialize cell weights

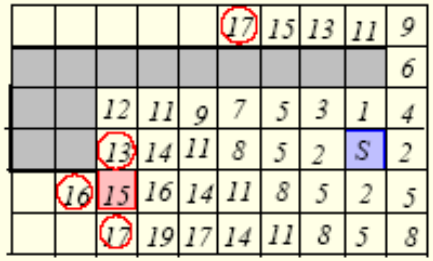


wave propagation

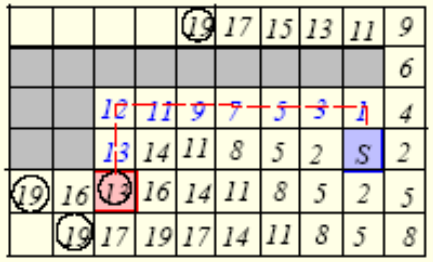




first wave reaches the target



finding other paths



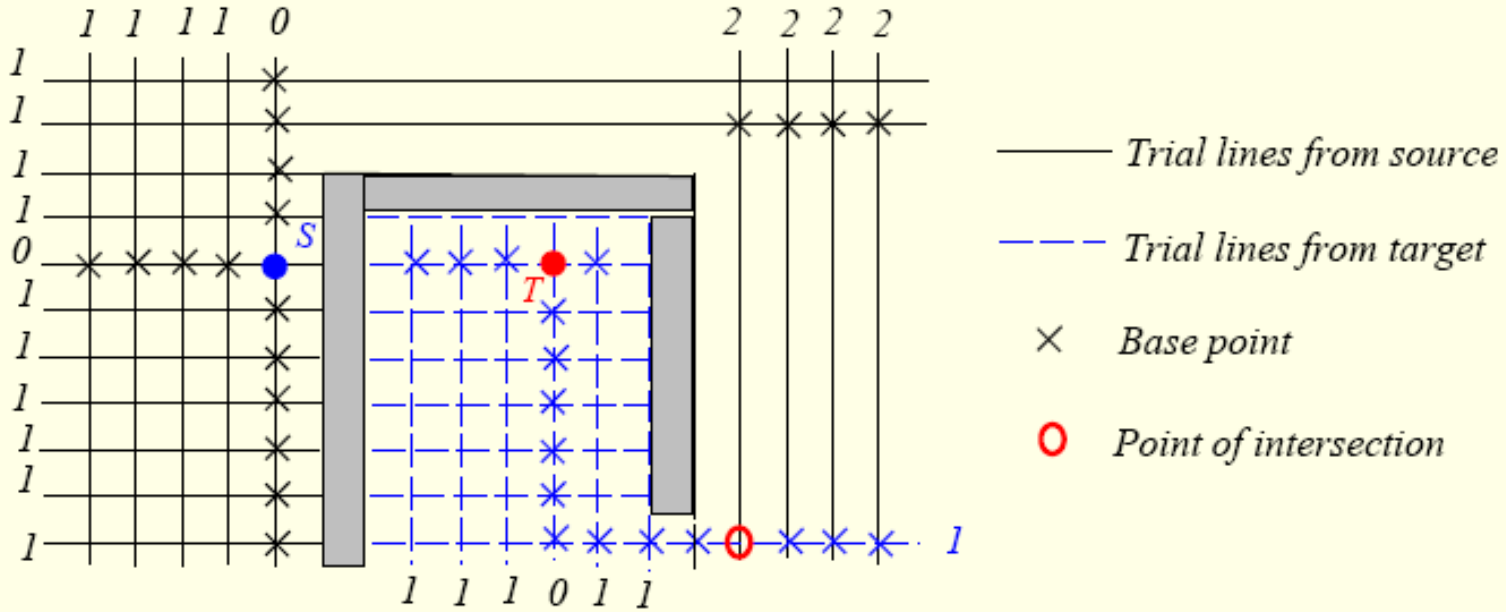
min−cost path found

Branch-and-bound

We can stop propagating a wavefront n with label >= shortest path length discovered so far. This is because the label of n's neighbors will always increase IF they are propagated from n. So we only need to propagate the node with label 12. We stop when there is no more wavefront to propagate.

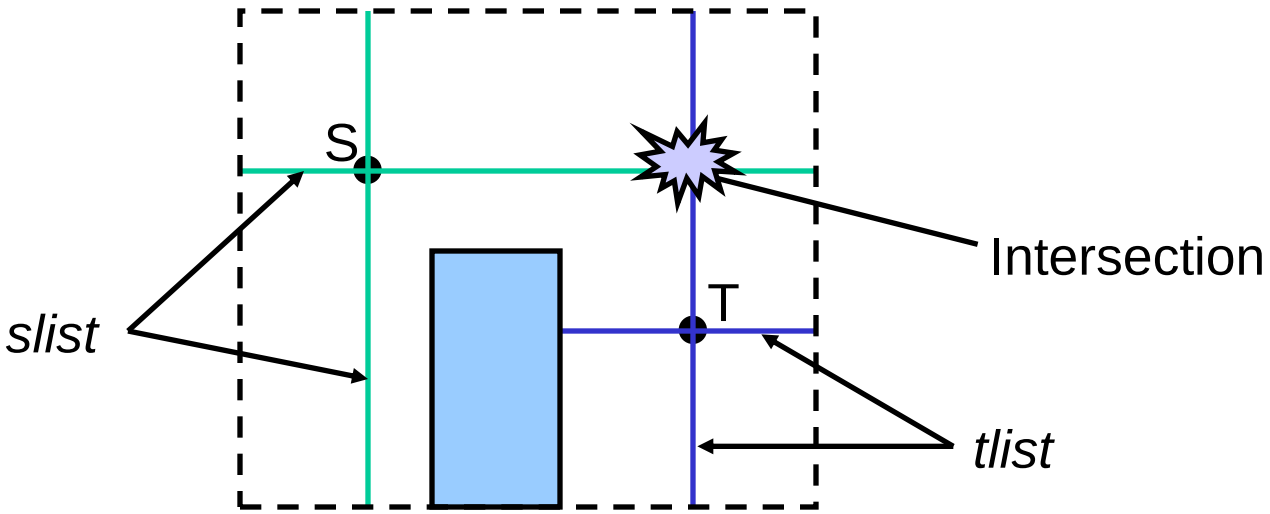
Mikami & Tabuchi's Algorithm

- Mikami & Tabuchi, "A computer program for optimal routing of printed circuit connectors," IFIP, H47, 1968.
- Every grid point is an escape point.



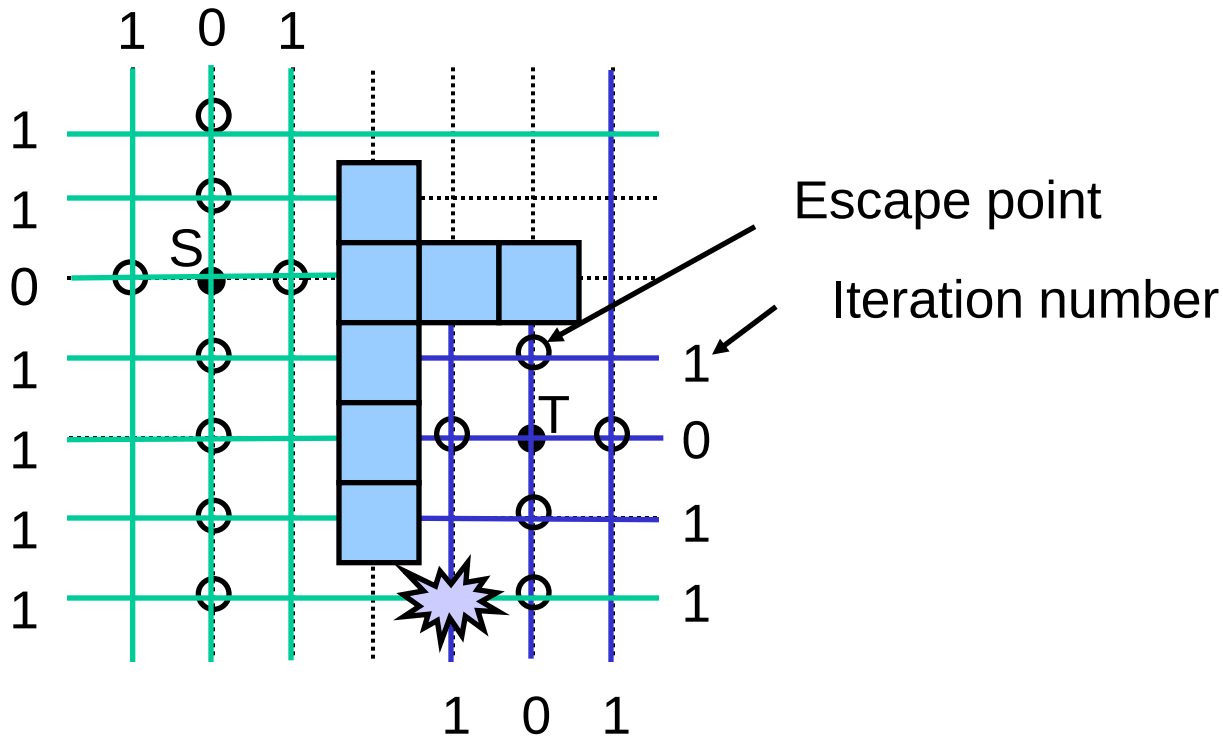
Line Probing

- Keep two lists of line segments, *slist* and *tlist*, for the source and the target respectively
- If a line segment from *slist* intersects with one from *tlist*, a route is found; else, new line segments are generated from the escape points



Line Probing

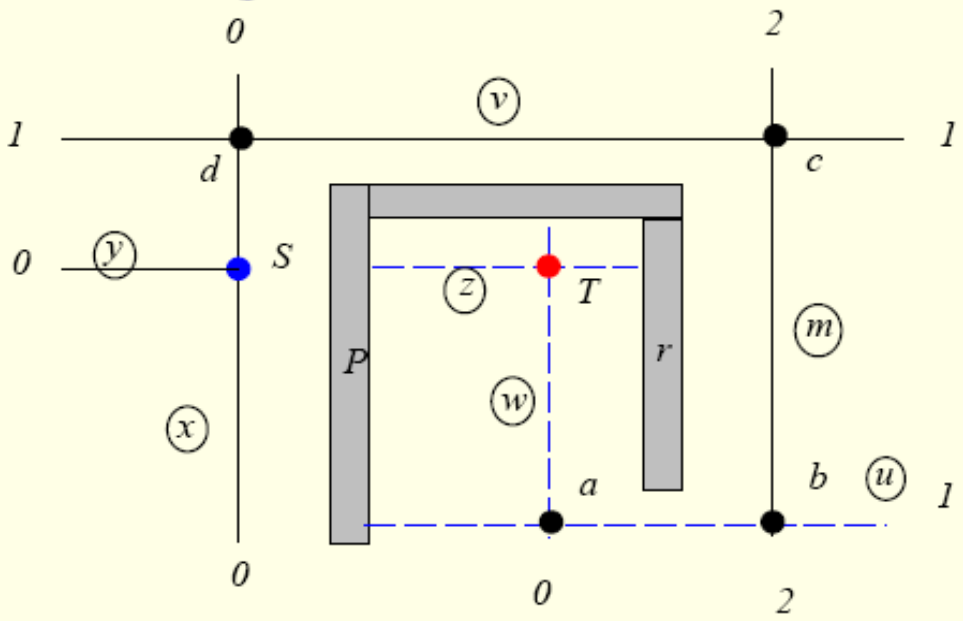
- We can use all the grid vertices on the line segments as escape points:



- Always find a path but may not be optimal

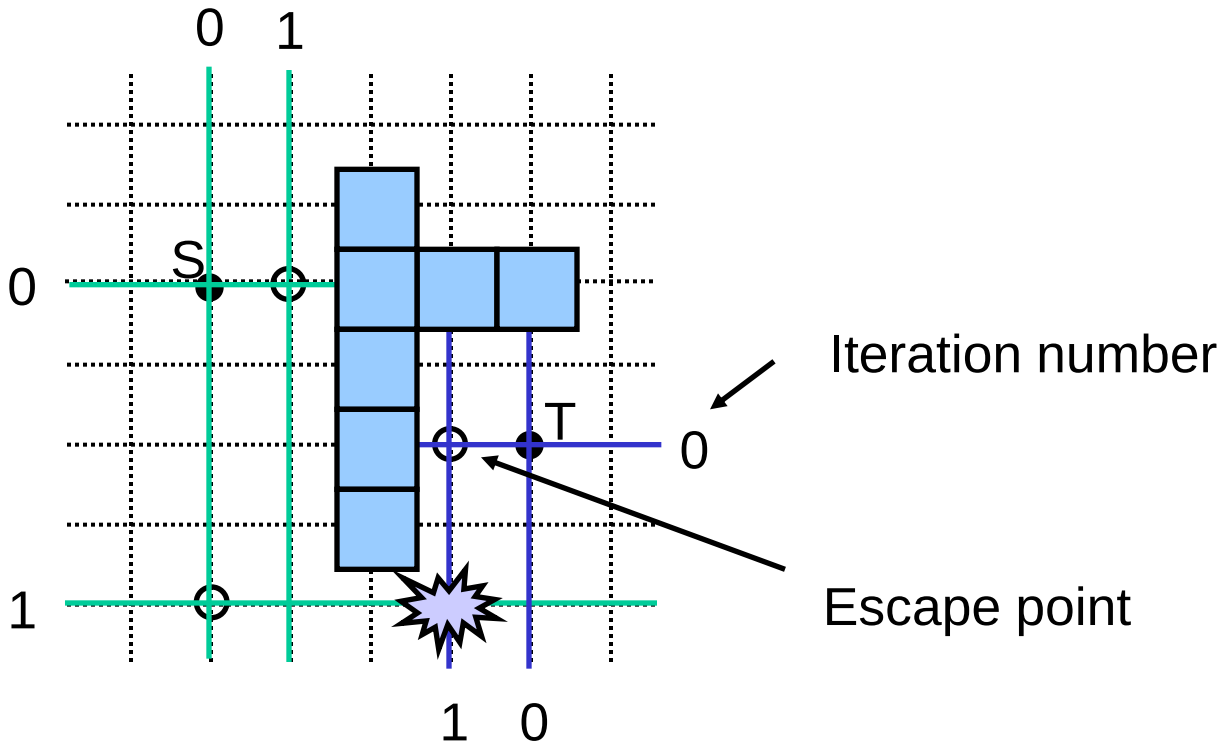
Hightower's Algorithm

- Hightower, "A solution to line-routing problem on the continuous plane," DAC-69.
- A single escape point on each line segment.
- If a line parallels to the blocked cells, the escape point is placed just past the endpoint of the segment.



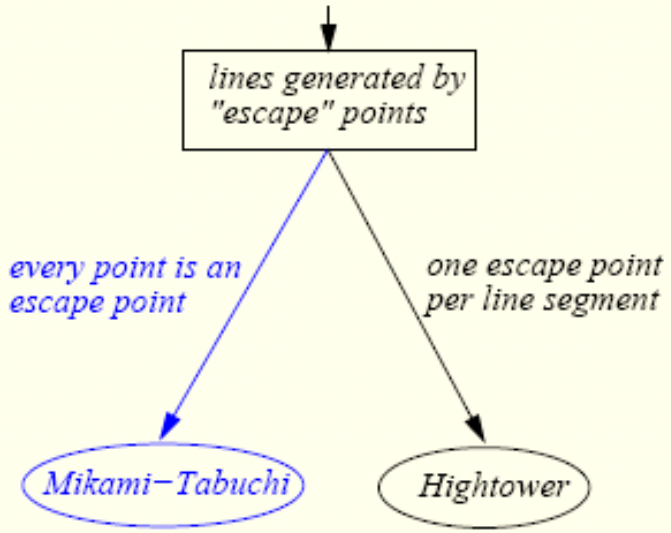
Line Probing

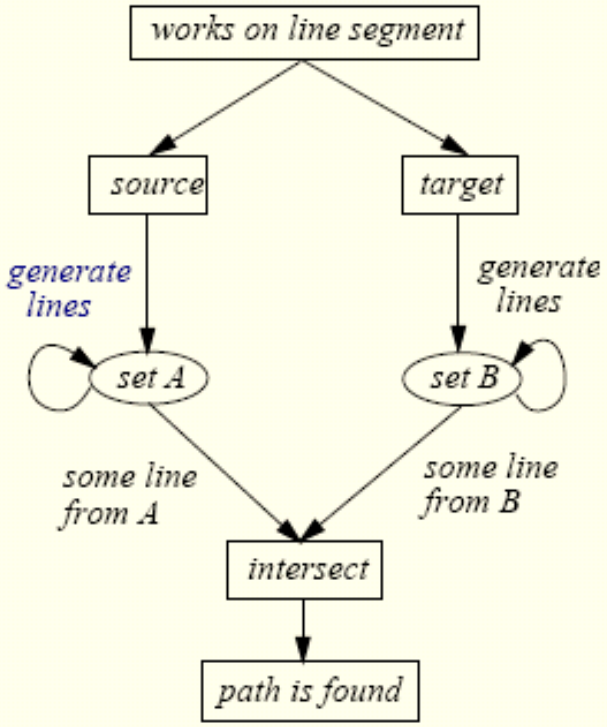
- We can pick just one escape point from each line segment



- May fail to find a path even if one exists

Features of Line-Search Algorithms





- Time and space complexities: $O(L)$, where L is the # of line segments generated.

Comparison of Algorithms

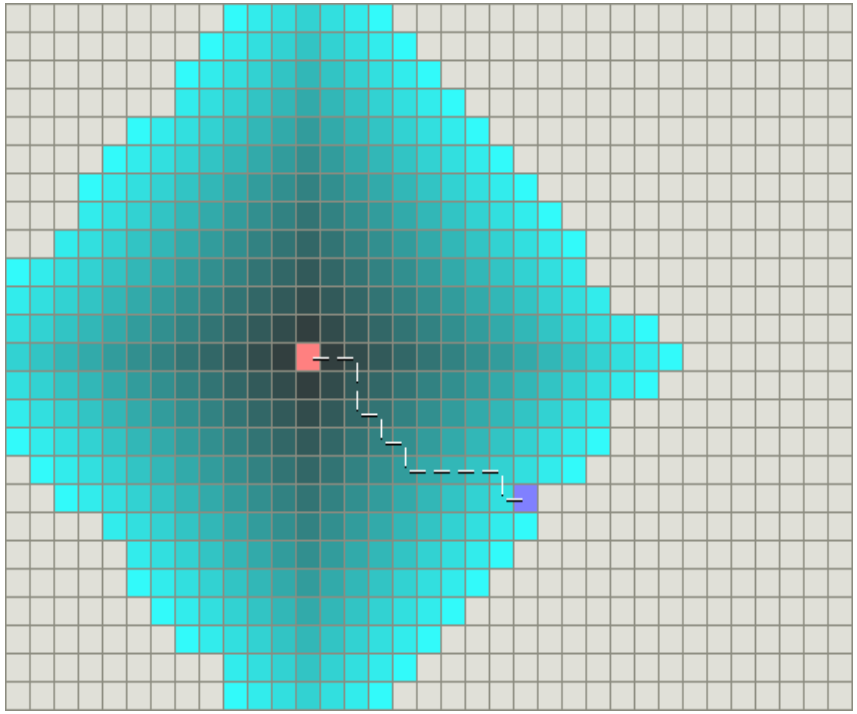
	Maze routing			Line search	
	Lee	Soukup	Hadlock	Mikami	Hightower
Time	$O(MN)$	$O(MN)$	$O(MN)$	$O(L)$	$O(L)$
Space	$O(MN)$	$O(MN)$	$O(MN)$	$O(L)$	$O(L)$
Finds path if one exists?	yes	yes	yes	yes	no
Is the path shortest?	yes	no	yes	no	no
Works on grids or lines?	grid	grid	grid	line	line

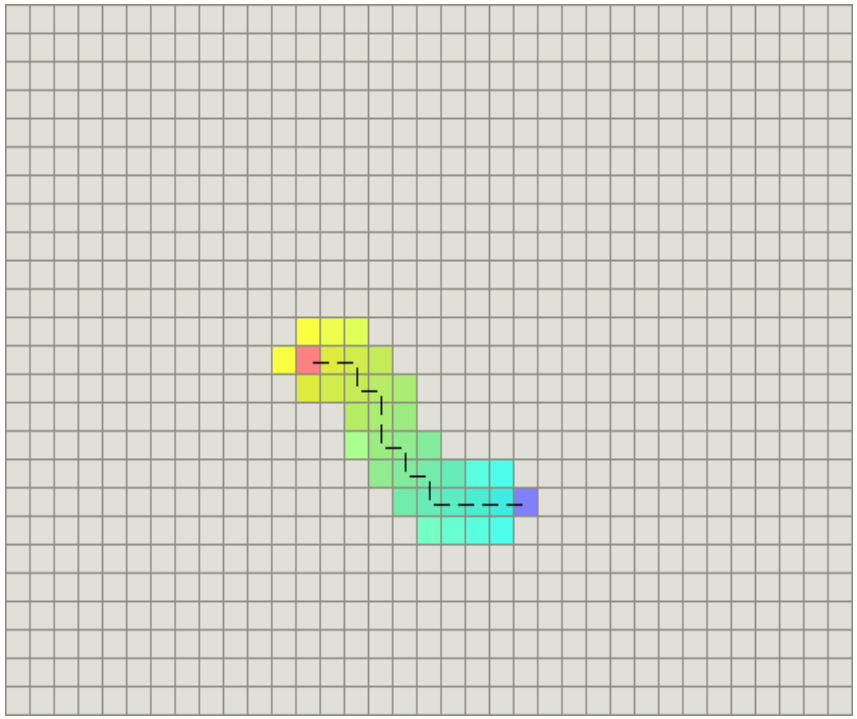
- Soukup, Mikami, and Hightower all adopt some sort of line-search operations $\Rightarrow$ cannot guarantee shortest paths.

BFS based Maze Routing (A*)

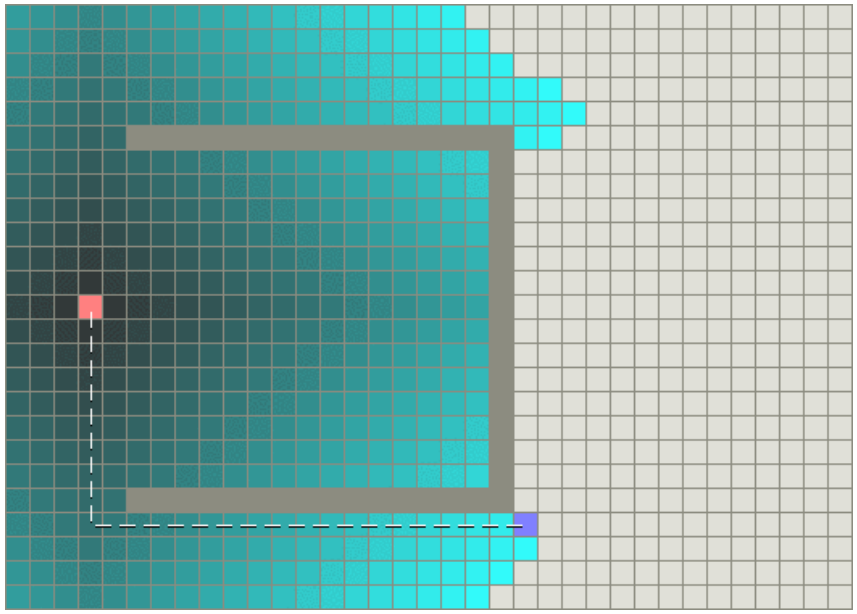
- Need to search whole space?
  - Guide the search to the goal explicitly
- A* search is faster if you need “good path”, not “perfect path”
  - Use priority queue
  - C(n) = F(n)+H(n)
    - F(n) is a computed cost from source to current location
    - H(n) is a predicted cost from current location to target
    - If H(n)=0, it becomes maze routing!
  - Optimal (shortest path) when $H(n) \leq H'(n)$ (no overestimation)
    - H’(n) is the exact cost
    - H(n)=0 never overestimates!

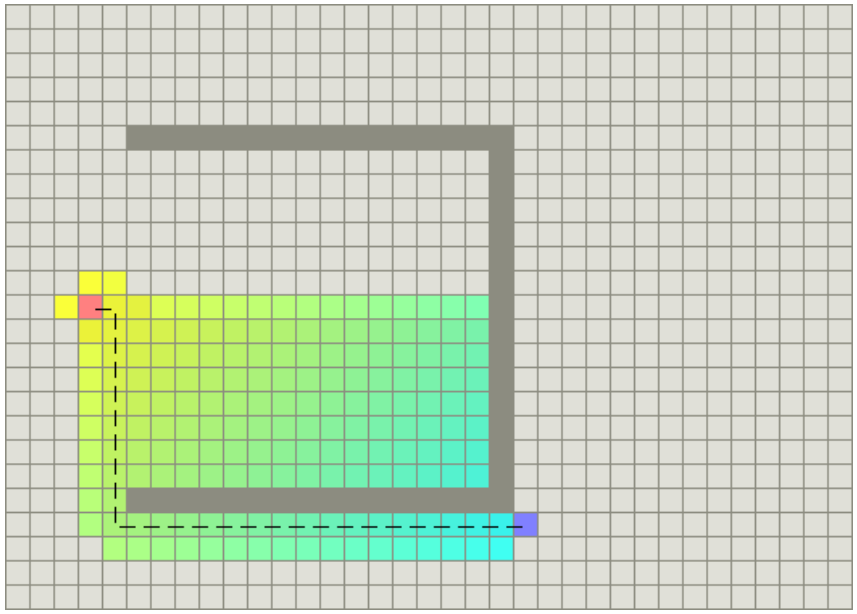
Maze vs A* routing (I)





Maze vs A* routing (II)





Shortest Path Based Algorithms

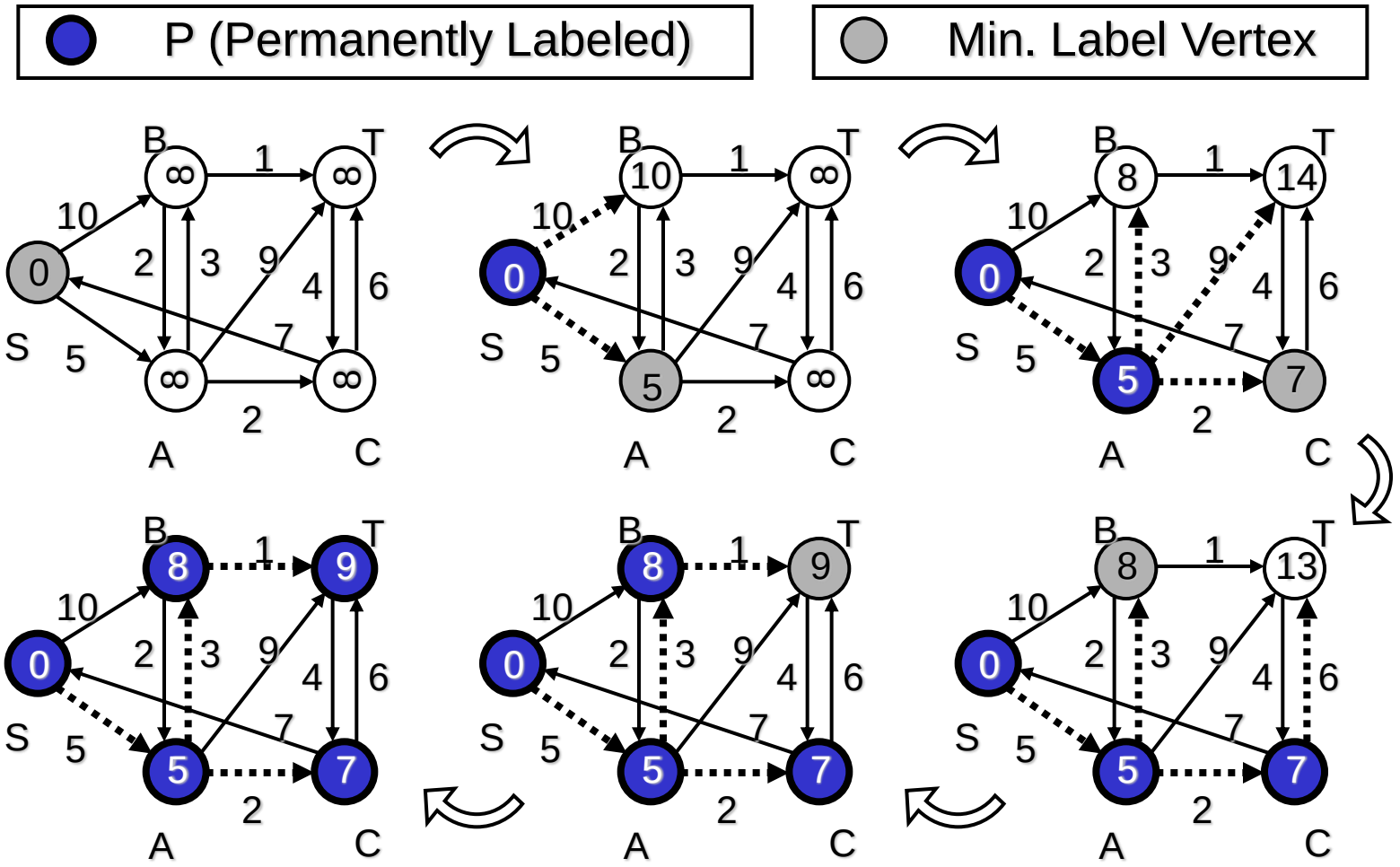
- For 2-terminal nets only
- Use Dijkstra's algorithm to find the shortest path between the source *s* and the sink *t* of a net
- Different from Maze Routing:
  - The graph need not be a rectangular grid
  - The edges need not be of unit length

Dijkstra's Shortest Path Algorithm

- Label of vertices = Shortest distance from S
- Let P be the set of permanently labeled vertices

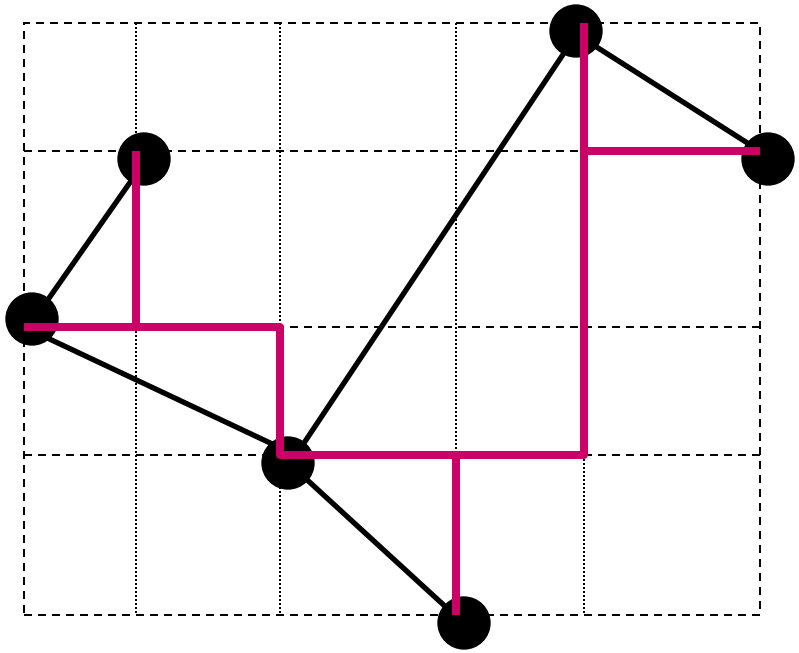
- Initially,
  - P = Empty Set.
  - Label of S = 0, Label of all other vertices = infinity
- While (T is not in P) do
  - Pick the vertex v with the min. label among all vertices not in P
  - Add v to P
  - Update the label for all neighbors of v

Dijkstra's Algorithm: Example



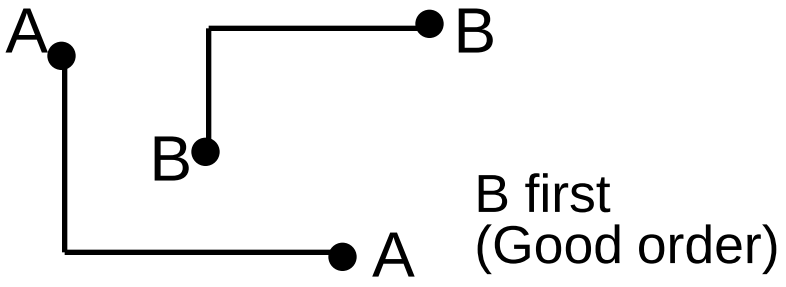
Steiner Tree Based Algorithms

- For multi-terminal nets
- Find Steiner tree instead of shortest path
- Construct a Steiner tree from the minimum spanning trees (MST)

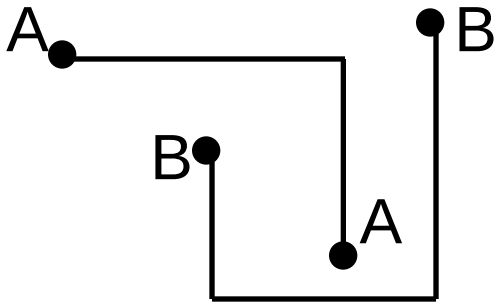


Net Ordering

- In sequential approach, we need some net ordering
- A bad net ordering will increase the total wire length, and may even prevent completion of routing for some circuits which are indeed routable



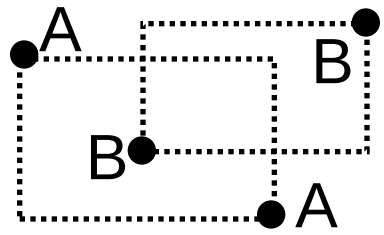
B first
(Good order)



A first
(Bad order)

Criteria for Net Ordering

- Criticality of net – critical nets first
- Estimated wire length – short nets first since they are less flexible
- Consider bounding rectangles (BR)

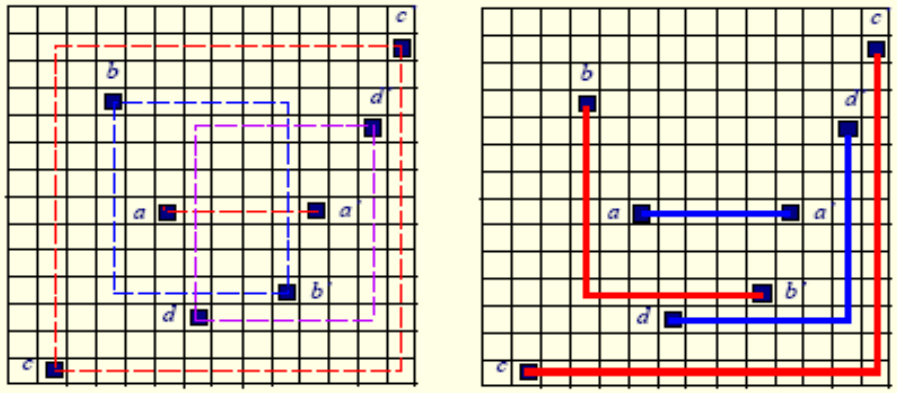


B is in A's BR

Which one should be routed first and why? (Note that this rule of thumb is not always applicable.)

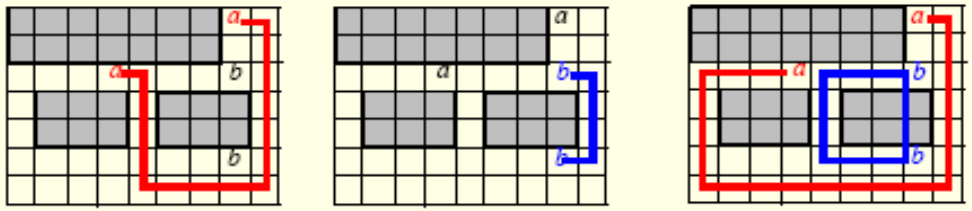
Net Ordering (cont'd)

- Order the nets in the ascending order of the # of pins within their bounding boxes.
- Order the nets in the ascending (or descending??) order of their lengths.
- Order the nets based on their timing criticality.



routing ordering: a (0) −> b (1) −> d (2) −> c (6)

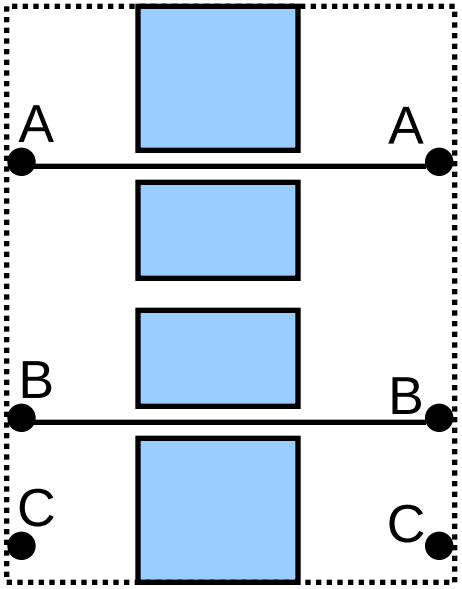
- A mutually intervening case:



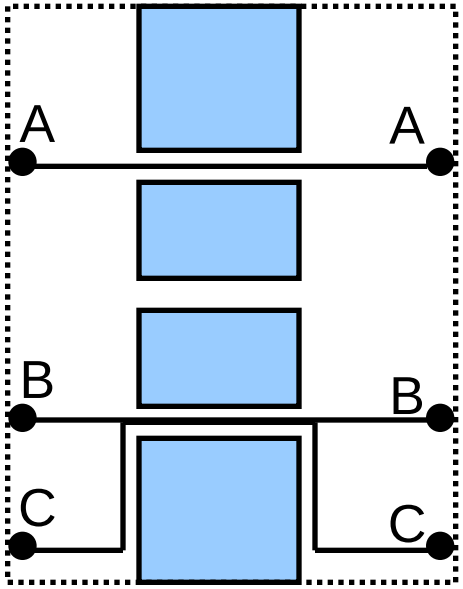
a prevents routing of b *b prevents routing of a* *a feasible routing*

Rip-up and Re-route

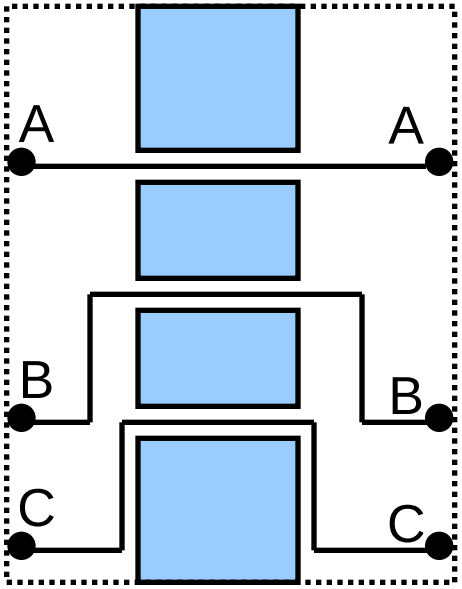
- It is impossible to get the optimal net ordering
- If some nets are failed to be routed, the rip-up and re-route technique can be applied:



Cannot route C



So rip-up B
and route C first



Finally route B

Concurrent Approach

- Consider all the nets simultaneously
- Formulate as an integer programming
- Given:

Nets	*Set of possible routing trees*
net 1	$T_{11}, T_{12}, \ldots\ldots, T_{1k_1}$
⋮	⋮
net *n*	$T_{n1}, T_{n2}, \ldots, T_{nk_n}$

L_{ij} = Total wire length of T_{ij}

C_e = Capacity of edge e

- Determine variable x_{ij} s.t. $x_{ij} = 1$ if T_{ij} is used

  $x_{ij} = 0$ otherwise

Integer Program Formulation

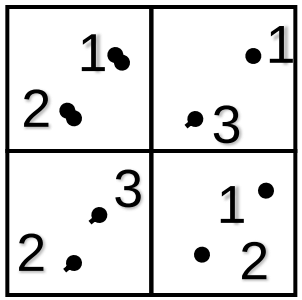
$$\text{Min.} \sum_{i=1}^{n} \sum_{j=1}^{k_i} L_{ij} \times x_{ij}$$

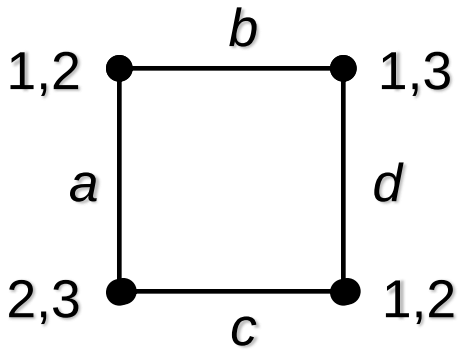
$$\text{s.t.} \quad \sum_{j=1}^{k_i} x_{ij} = 1 \quad \text{for all } i = 1, \ldots, n$$

$$\sum_{i,j \text{ s.t.} e \in T_{ij}} x_{ij} \leq C_e \quad \text{for all edge } e$$

$$x_{ij} = 0 \text{ or } 1 \quad \forall i, j$$

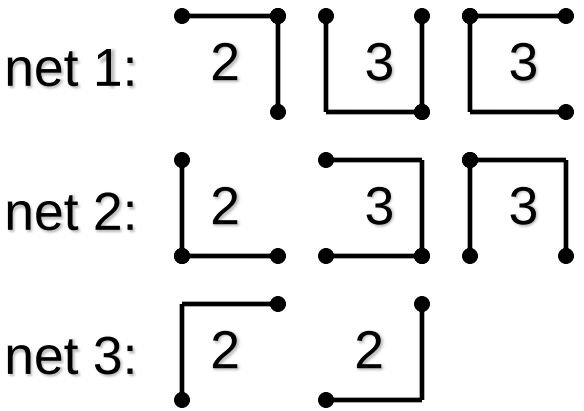
Concurrent Approach: Example





$C_a = C_b = C_c = C_d = 2$

Possible trees:



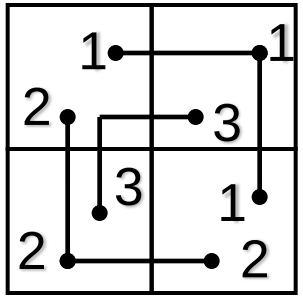
$$\text{Min. } 2x_{11} + 3x_{12} + 3x_{13} + 2x_{21} + 3x_{22} + 3x_{23} + 2x_{31} + 2x_{32}$$

$$\text{s.t.} \begin{cases} x_{11} + x_{12} + x_{13} = 1; \\ x_{21} + x_{22} + x_{23} = 1; \\ x_{31} + x_{32} = 1; \\ x_{ij} = 0 \text{ or } 1 \quad \forall i, j; \end{cases}$$

What are the constraints for edge capacity?

$$x_{12} + x_{13} + x_{21} + x_{23} + x_{31} \le C_a$$

Solution



Integer Programming Approach

- Standard techniques to solve ILP
- No net ordering → Give global optimum
- Can be extremely slow, especially for large problems
- To make it faster, a fewer choices of routing trees for each net can be used. May make the problem infeasible or give a bad solution
- Determining a good set of choices of routing trees is a hard problem by itself

Hierarchical Approach to Speed Up Integer Programming Formulation For Global Routing

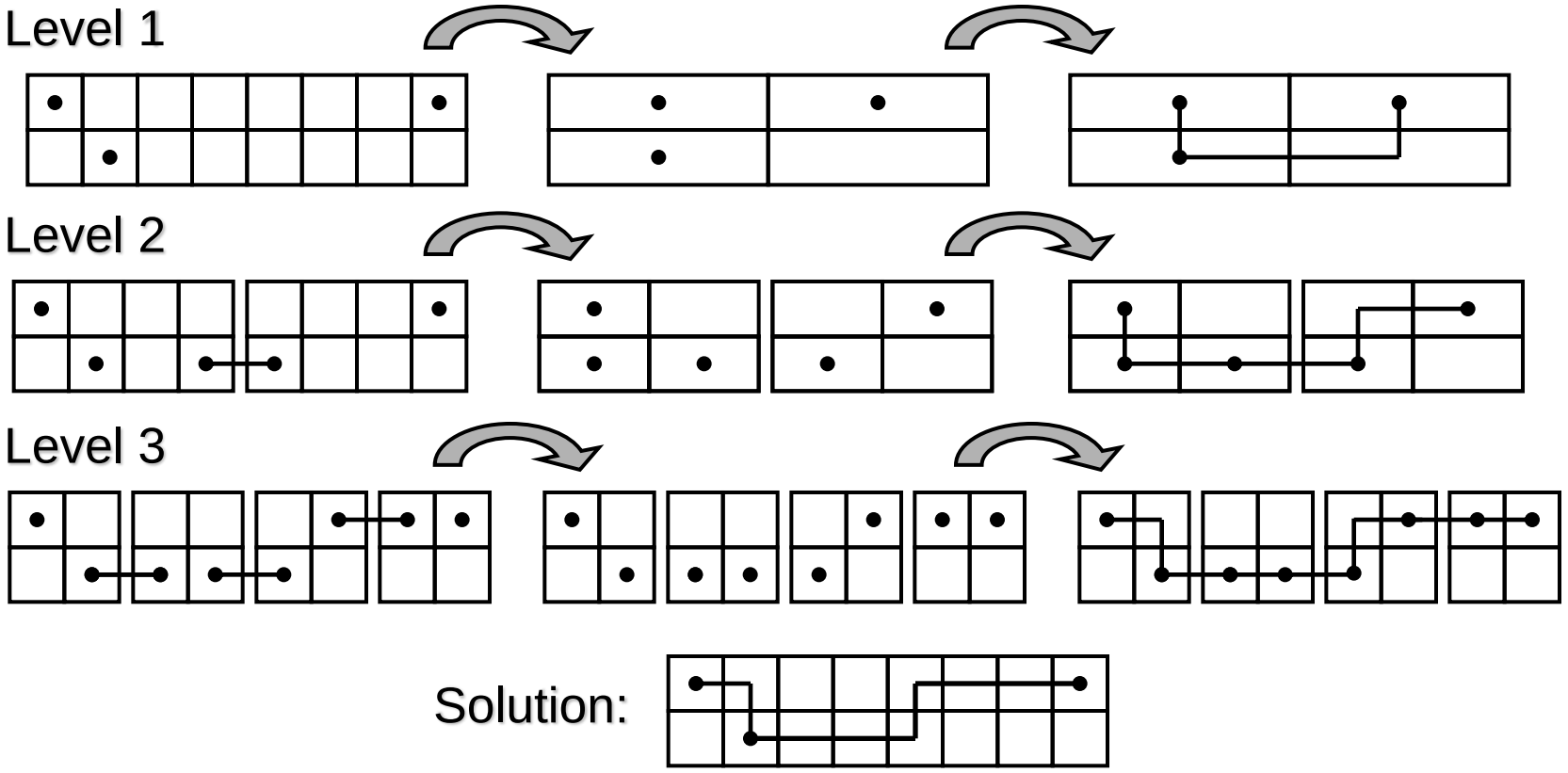
M. Burstein and R. Pelavin,
“Hierarchical Wire Routing”,
IEEE TCAD, vol. CAD-2,
pages 223-234, Oct. 1983.

Hierarchical Approach

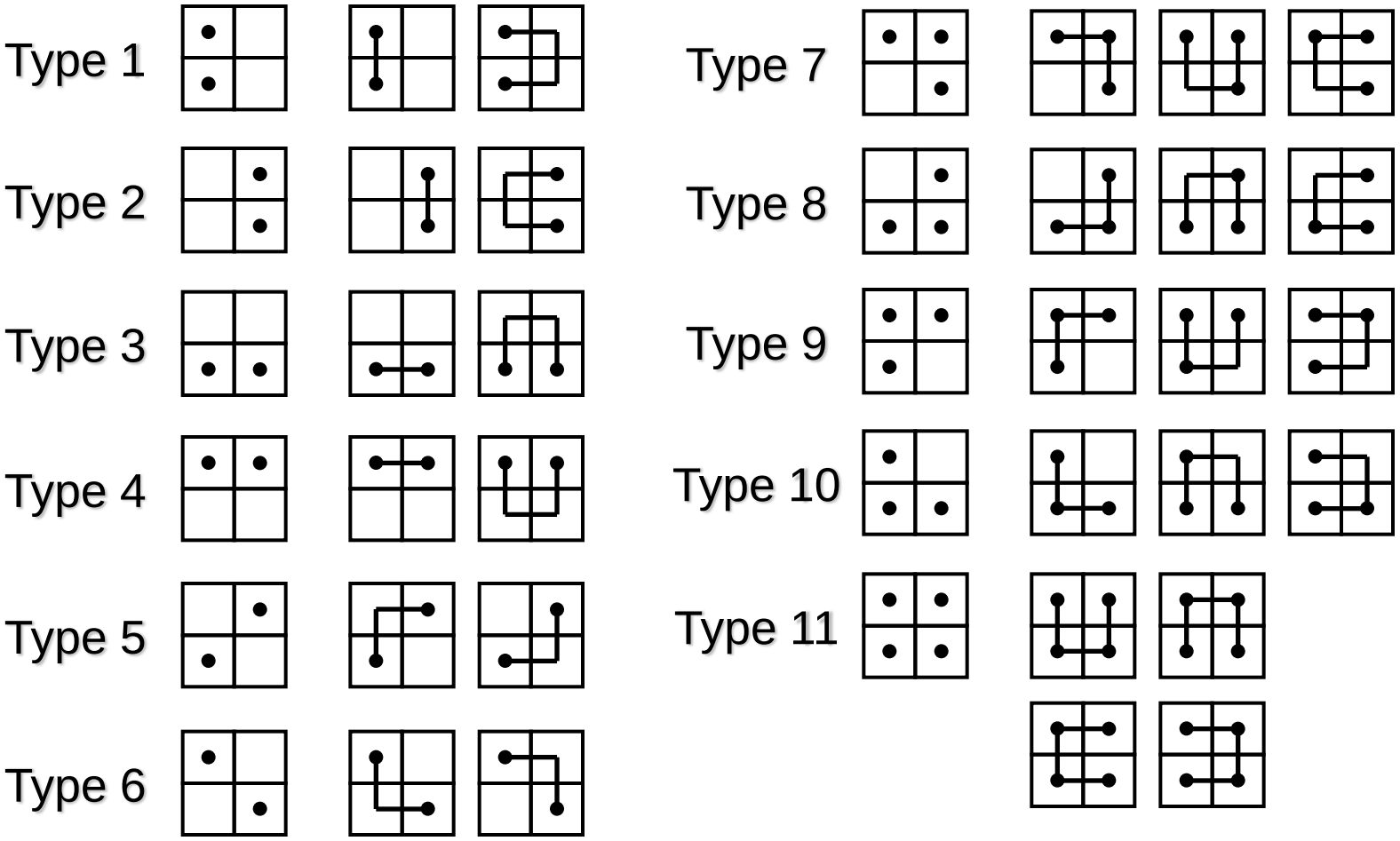
- Large Integer Programs are difficult to solve
- Hierarchical Approach reduces global routing to routing problems on a 2×2 grid
- Decompose recursively in a top-down fashion
- Those 2×2 routing problems can be solved optimally by integer programming formulation

Hierachical Approach: Example

- Solving a 2 × n routing problem hierarchically



Types of 2×2 Routing Problems



Objective Function of 2×2 Routing

- Possible Routing Trees:

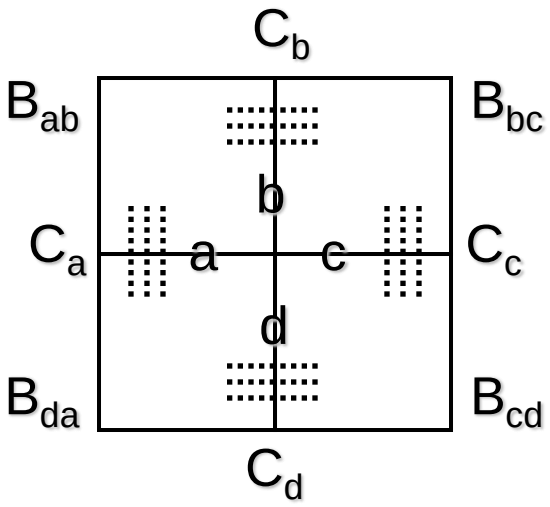
  $T_{1,1}, T_{1,2}, T_{2,1}, T_{2,2}, \ldots\ldots, T_{11,1}, \ldots, T_{11,4}$

- # of nets of each type: $n_1, \ldots, n_{11}$
- Determine
  - x_{ij}: # of type-i nets using T_{ij} for routing.
  - y_i: # of type-i nets that fails to route

$$y_i + \sum_j x_{ij} = n_i \quad i = 1, \ldots, 11$$

$$\text{Minimize} \quad \sum_{i=1}^{11} y_i$$

Constraints of 2×2 Routing



Constraints on Edge Capacity:

$$\sum_{i,j \text{ s.t.} a \in T_{ij}} x_{ij} \le C_a$$

$$\sum_{i,j \text{ s.t.} b \in T_{ij}} x_{ij} \le C_b$$

$$\sum_{i,j \text{ s.t.} c \in T_{ij}} x_{ij} \le C_c$$

$$\sum_{i,j \text{ s.t.} d \in T_{ij}} x_{ij} \le C_d$$

Constraints on # of Bends in a Region:

$$\sum_{i,j \text{ s.t.} T_{ij} \text{ has a bend in region } ab} x_{ij} \le B_{ab}$$

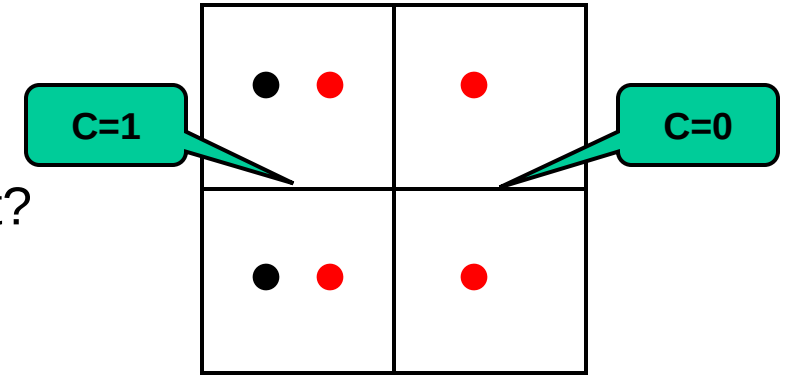
$$\sum_{i,j \text{ s.t.} T_{ij} \text{ has a bend in region } bc} x_{ij} \le B_{bc}$$

$$\sum_{i,j \text{ s.t.} T_{ij} \text{ has a bend in region } cd} x_{ij} \le B_{cd}$$

$$\sum_{i,j \text{ s.t.} T_{ij} \text{ has a bend in region } da} x_{ij} \le B_{da}$$

Pop Quiz

- If there are two nets, one with 2 pins, the other with 4 pins with a zero capacity edge
  - What is going to be the result?

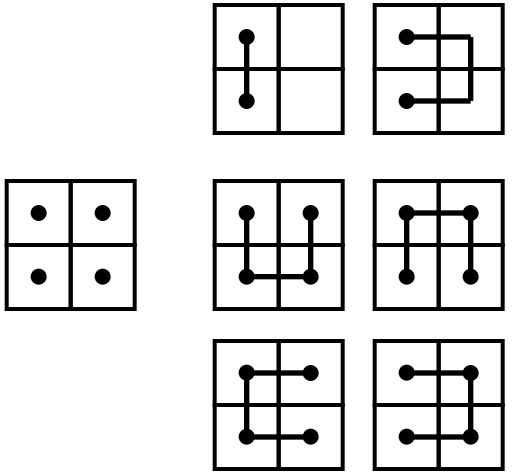


$$y_i + \sum_j x_{ij} = n_i \quad i = 1,\ldots,11$$

$$\text{Minimize} \quad \sum_{i=1}^{11} y_i$$

Type 1

Type 11



ILP Formulation of 2×2 Routing

$$\text{Min.} \quad \sum_{i=1}^{11} y_i$$

$$\text{s.t.} \quad y_i + \sum_j x_{ij} = n_i \quad i = 1, \ldots, 11$$

$$x_{ij} \geq 0,\ y_i \geq 0 \quad \forall i, j$$

$$\sum_{i,j \text{ s.t.} a \in T_{ij}} x_{ij} \leq C_a \qquad \sum_{i,j \text{ s.t.} T_{ij} \text{ has a bend in region } ab} x_{ij} \leq B_{ab}$$

$$\sum_{i,j \text{ s.t.} b \in T_{ij}} x_{ij} \leq C_b \qquad \sum_{i,j \text{ s.t.} T_{ij} \text{ has a bend in region } bc} x_{ij} \leq B_{bc}$$

$$\sum_{i,j \text{ s.t.} c \in T_{ij}} x_{ij} \leq C_c \qquad \sum_{i,j \text{ s.t.} T_{ij} \text{ has a bend in region } cd} x_{ij} \leq B_{cd}$$

$$\sum_{i,j \text{ s.t.} d \in T_{ij}} x_{ij} \leq C_d \qquad \sum_{i,j \text{ s.t.} T_{ij} \text{ has a bend in region } da} x_{ij} \leq B_{da}$$

- Only 39 variables (28 x_{ij} and 11 y_i) and 19 constraints (plus 38 non-negative constrains)
- Problems of this size are usually not too difficult to solve

